Vietnam National University, Ho Chi Minh City

University of Science

Faculty of Information Technology

Project 02: Futoshiki

Solving with First-Order Logic & Inference Algorithms

Instructor: Dr. Lê Ngọc Thành
MSc. Lê Nhựt Nam

List of students:

Student 1:	Hoàng Trung Hiếu	*Student 2:*	Lương Hưng Phát
Student ID 1	24127041	*Student ID 2:*	24127298
Student 3:	Nguyễn Thành Đạt	*Student 4:*	Nguyễn Minh Khoa
Student ID 3:	24127021	*Student ID 4:*	24127188

Ho Chi Minh City

March 2026

Contents

List of Algorithms **vi**

1 Introduction and Problem Formulation **1**

1.1 Project Overview 1

1.2 Objectives 1

1.3 Scope and Deliverables 2

1.3.1 Puzzle Sizes and Input/Output 2

1.3.2 Algorithms Covered 2

1.4 Evaluation Criteria 3

2 FOL Formalization **4**

2.1 Predicates and vocabulary 4

2.1.1 Vocabulary 4

2.1.2 Core Axioms 4

2.1.3 Conversion to Conjunctive Normal Form (CNF) 5

2.1.4 Derive the CNF(Skolemization, clausal form) for N = 4 7

2.2 Knowledge base implementation 15

2.2.1 KB Architecture 15

2.2.2 FOL builder 15

2.2.3 Why All rules conclude FALSE 17

2.2.4 Why symbolic FOL (facts, rules) + separate grounding 18

3 Unification and Binding Mechanism **20**

3.1 Core idea 20

3.2 Process overview 20

3.3 Function envolved 20

3.3.1 _apply_bindings(pattern, bindings) 20

3.3.2 _unify_one(pattern, fact, bindings) 21

3.3.3 _unify_all(premises, facts, bindings) 21

3.3.4 _derive_false(rules, facts) 22

3.4 Limitations 23

4 Forward Chaining **24**
4.1 Overview. 24
4.2 Key Design: Domain-Based Elimination. 24
4.3 Limitations of Forward Chaining. 27
4.4 Illustrative Example 28

5 Backward Chaining **35**
5.1 Algorithm Description 35
5.1.1 SLD Refutation 35
5.1.2 Operating Principle 35
5.1.3 Pseudocode 36
5.2 Limitations 37
5.3 Illustrative Example 37
5.3.1 Initial State 37
5.3.2 Step-by-Step Execution 38
5.3.3 Final Solution 41
5.4 Experimental Results 42
5.4.1 Performance Data 42
5.4.2 Scalability and Time Complexity Analysis 42
5.4.3 Impact of Constraint Density 42
5.4.4 Memory Management 43

6 A* Search & Heuristic Analysis **44**
6.1 State Space and Problem Formulation 44
6.2 Heuristic Functions 44
6.2.1 Remaining Unassigned Cells (h_1) 44
6.2.2 Inequality Chain Coverage (h_2) 46
6.2.3 Combined Heuristic (h_3) 49
6.3 Pruning Methods 50
6.3.1 Forward Chaining Pruning 50
6.3.2 AC-3 Pruning 51
6.3.3 Why not implement FOL in AC-3? 52
6.4 Core Algorithm Design 54
6.4.1 Operation 54
6.4.2 Illustrative Example 57
6.5 Experimental Result 62
6.5.1 Performance Data 62
6.5.2 Pruning Dominates Runtime; Heuristic Choice Is Secondary . . 62
6.5.3 FC Runtime Grows with Difficulty, AC-3 Runtime Shrinks . . . 63

6.5.4 Pruner Determines Memory Tier, Heuristic Is Invisible at Small Sizes . . . 63

7 Comparison & Analysis **64**
7.1 Experimental Setup . . . 64
7.1.1 Test Suite . . . 64
7.2 Runtime Comparison . . . 65
7.2.1 Small Grid Scale . . . 65
7.2.2 Large Grid Scale (7×7) . . . 67
7.3 Memory Comparison . . . 68
7.3.1 Small Grid Scale . . . 68
7.3.2 Large Grid Scale (7×7) . . . 70
7.4 FC Stall Behavior Investigation . . . 73
7.4.1 Performance Data . . . 73
7.4.2 The Stall Problem and How Each Variant Resolves It . . . 73
7.4.3 Runtime Comparison Between the Two Variants . . . 74
7.4.4 Heuristic Guidance Versus MRV on Stalling Instances . . . 74
7.4.5 Memory Cost of Extended Search . . . 74
7.4.6 Improve A* Runtime . . . 75

8 Graphical User Interface **76**
8.1 Design Overview . . . 76
8.2 Detailed Instructions . . . 77

9 Task Assignment and Self-Evaluation **80**
9.1 Task Assignment . . . 80
9.1.1 Work Assignment Table . . . 80
9.1.2 Self-Evaluation of Project Levels . . . 82

List of Figures

6.1 Runtime comparison of A* variants across puzzle sizes and difficulties . 62
6.2 Memory comparison of A* variants across puzzle sizes and difficulties . 63

7.1 Runtime comparison of all algorithms across puzzle sizes and difficulty levels 66
7.2 Runtime comparison of remaining algorithms on the 7×7 instance . . 68
7.3 Memory comparison of all algorithms on small grid instances 69
7.4 Memory comparison of remaining algorithms on the 7×7 instance . . . 71
7.5 Runtime & Memory comparison of FOL based solution for instances that stalled FC 73

8.1 The application interface 76
8.2 The grid during play 77
8.3 The grid after solved 78
8.4 Statistics and history log example 79

List of Tables

5.1 Performance results of backward chaining on Futoshiki instances 42

7.1 Test suite used in the experiments 65
7.2 FOL based algorithms omitted from detailed 7×7 runtime statistics . 67
7.3 FOL based algorithms omitted from detailed 7×7 memory statistics . 71

9.1 Team Member Information and Work Assignment 81
9.2 Self-Evaluation of Completion Rate at Each Project Level 82

List of Algorithms

Chapter 1

Introduction and Problem Formulation

1.1 Project Overview

This project studies **Futoshiki**, a logic based number placement puzzle, through the lens of **First-Order Logic (FOL)** and inference algorithms. A puzzle of size $N \times N$ requires filling each cell with a value in $\{1, \ldots, N\}$ while satisfying:

- (1) Row uniqueness.
- (2) Column uniqueness.
- (3) Fixed prefilled.
- (4) Inequality constraints between adjacent cells.

The core goal of this project is to bridge theory and implementation: *formalize the puzzle in FOL, transform logic into machine solvable representations, implement multiple reasoning/search algorithms from scratch, and compare their behavior on puzzles of different sizes.*

1.2 Objectives

The primary objectives are:

(1) **Logical Modeling**: Define a clean FOL vocabulary for Futoshiki and derive a complete axiom system.

(2) **Knowledge Base Construction**: Ground axioms for concrete puzzle sizes and convert them into implicit forms (including CNF).

(3) **Algorithm Implementation**: Implement and evaluate multiple solvers (forward chaining, backward chaining, backtracking, and A*) from scratch.

(4) **Heuristic Design**: Build and justify admissible heuristics for A* on partial assignments.

(5) **Comparison**: Analyze solution quality, runtime,and expansion across puzzle sizes.

1.3 Scope and Deliverables

1.3.1 Puzzle Sizes and Input/Output

The implementation targets multiple grid sizes (4×4, 5×5, 6×6, 7×7, and 9×9) with standardized text based input files containing:

- Grid values (0 for empty cells; 1 to N for given clues).
- Horizontal inequalities,
- Vertical inequalities.

For each input instance, the system outputs a fully solved grid while preserving inequality symbols for readability.

1.3.2 Algorithms Covered

Following the project specification, the solver suite includes:

- **Forward Chaining**: Data driven fact propagation over rule instances.
- **Backward Chaining (SLD)**: Goal driven querying of cell assignments.
- **A* Search**: Heuristic search over partial assignments with consistency checks.
- **Backtracking / Brute force Baselines**: Reference methods for comparison.

In addition, the report includes manual derivation for a 3×3 case (axioms, grounding intuition, and CNF oriented transformation steps).

1.4 Evaluation Criteria

To compare algorithms fairly, experiments are evaluated under consistent metrics:

- **Correctness**: every returned grid satisfies all Futoshiki constraints.
- **Efficiency**: expansion/inference effort, runtime and memory usage.
- **Robustness**: performance stability across diverse puzzle instances.

Chapter 2

FOL Formalization

2.1 Predicates and vocabulary

2.1.1 Vocabulary

A compact FOL vocabulary is used to model puzzle semantics:

- Val(i, j, v): cell (i, j) is assigned value v.
- Given(i, j, v): cell (i, j) is pre-filled with value v.
- LessH(i, j), GreaterH(i, j): horizontal inequality between (i, j) and $(i, j + 1)$.
- LessV(i, j), GreaterV(i, j): vertical inequality between (i, j) and $(i + 1, j)$.
- Less(v_1, v_2): arithmetic ordering relation $v_1 < v_2$.

Indices and values range over $\{1, \ldots, N\}$.

2.1.2 Core Axioms

A1 (At least one value per cell):

$$\forall i \forall j \; \exists v \; Val(i, j, v)$$

A2 (At most one value per cell):

$$\forall i \forall j \forall v_1 \forall v_2 \; \Big(Val(i, j, v_1) \wedge Val(i, j, v_2) \rightarrow v_1 = v_2 \Big)$$

A3 (Row uniqueness):

$$\forall i \forall j_1 \forall j_2 \forall v \; \Big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \rightarrow \bot \Big)$$

A4 (Horizontal less-than constraint):

$$\forall i \forall j \forall v_1 \forall v_2 \left(LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \rightarrow Less(v_1,v_2) \right)$$

A5 (Given values from the start):

$$\forall i \forall j \forall v \left(Given(i,j,v) \rightarrow Val(i,j,v) \right)$$

A6 (Column uniqueness):

$$\forall j \forall i_1 \forall i_2 \forall v \left(Val(i_1,j,v) \wedge Val(i_2,j,v) \wedge i_1 \neq i_2 \rightarrow \bot \right)$$

A7 (Vertical less-than constraint):

$$\forall i \forall j \forall v_1 \forall v_2 \left(LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2) \rightarrow Less(v_1,v_2) \right)$$

A8 (Horizontal greater-than constraint):

$$\forall i \forall j \forall v_1 \forall v_2 \left(GreaterH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \rightarrow Less(v_2,v_1) \right)$$

A9 (Vertical greater-than constraint):

$$\forall i \forall j \forall v_1 \forall v_2 \left(GreaterV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2) \rightarrow Less(v_2,v_1) \right)$$

A10 (Domain constraint):

$$\forall i \forall j \forall v \left(Val(i,j,v) \rightarrow (1 \leq v \leq N) \right)$$

2.1.3 Conversion to Conjunctive Normal Form (CNF)

CNF Derivation of Axiom A1

Original:

$$\forall i \forall j \exists v \; Val(i,j,v)$$

Step 1 (Skolemization):
Replace v with Skolem function $f(i,j)$, drop existential quantifier $\exists v$:

$$\forall i \forall j \; Val(i,j,f(i,j))$$

Step 2 (Drop universal quantifiers $\forall i,j$):

$$Val(i,j,f(i,j))$$

CNF Derivation of Axiom A2

Original:

$$\forall i \forall j \forall v_1 \forall v_2 \ (Val(i,j,v_1) \wedge Val(i,j,v_2) \rightarrow v_1 = v_2)$$

Step 1 (Remove implication):

$$\forall i \forall j \forall v_1 \forall v_2 \ \neg(Val(i,j,v_1) \wedge Val(i,j,v_2)) \vee (v_1 = v_2)$$

Step 2 (De Morgan):

$$\forall i \forall j \forall v_1 \forall v_2 \ \neg Val(i,j,v_1) \vee \neg Val(i,j,v_2) \vee (v_1 = v_2)$$

Step 3 (Drop universal quantifiers):

$$\neg Val(i,j,v_1) \vee \neg Val(i,j,v_2) \vee (v_1 = v_2)$$

CNF Derivation of Axiom A3

Original:

$$\forall i \forall j_1 \forall j_2 \forall v \ \Big(Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2 \rightarrow \bot\Big)$$

Step 1(Drop universal quantifiers):

$$Val(i,j_1,v) \wedge Val(i,j_2,v) \wedge j_1 \neq j_2 \rightarrow \bot$$

Step 2(Remove implication):

$$\neg Val(i,j_1,v) \vee \neg Val(i,j_2,v) \vee (j_1 = j_2)$$

CNF Derivation of Axiom A4

Original:

$$\forall i \forall j \forall v_1 \forall v_2 \ \Big(LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \rightarrow Less(v_1,v_2)\Big)$$

Step 1(Drop universal quantifiers):

$$LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \rightarrow Less(v_1,v_2)$$

Step 2(Remove implication):

$$\neg LessH(i,j) \vee \neg Val(i,j,v_1) \vee \neg Val(i,j+1,v_2) \vee Less(v_1,v_2)$$

CNF Derivation of Axiom A5

Original:

$$\forall i \forall j \forall v \left(Given(i,j,v) \rightarrow Val(i,j,v) \right)$$

Step 1(Drop universal quantifiers):

$$Given(i,j,v) \rightarrow Val(i,j,v)$$

Step 2(Remove implication):

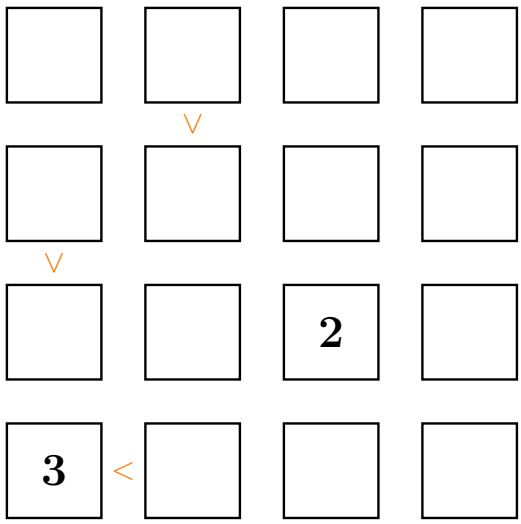
$$\neg Given(i,j,v) \vee Val(i,j,v)$$

2.1.4 Derive the CNF(Skolemization, clausal form) for N = 4

For this instance, we fix the finite domains as:

$$Rows = Cols = Values = \{1,2,3,4\}$$

We choose this puzzle:



From the puzzle, the pre-filled clues are:

$$Given(3,3,2), \qquad Given(4,1,3).$$

The horizontal inequality fact is:

$$LessH(4,1).$$

The vertical inequality facts are:

$$GreaterV(1,2), \qquad GreaterV(2,1).$$

The arithmetic background facts for the value domain are:

$$Less(1,2),\ Less(1,3),\ Less(1,4),\ Less(2,3),\ Less(2,4),\ Less(3,4).$$

For convenience, we define the unordered pairs of distinct values:

$$\mathcal{V}_2 = \{(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)\}.$$

Also define the unordered pairs of distinct indices:

$$\mathcal{P}_2 = \{(1,2),(1,3),(1,4),(2,3),(2,4),(3,4)\}.$$

A1. At least one value per cell

The original axiom is

$$\forall i \forall j\ \exists v\ Val(i,j,v).$$

A classical Skolemization gives

$$\forall i \forall j\ Val(i,j,f(i,j)),$$

where $f(i,j)$ is a Skolem function.

Since the value domain is finite and known, in the grounded KB we replace the existential quantifier by a finite disjunction:

$$Val(i,j,1) \vee Val(i,j,2) \vee Val(i,j,3) \vee Val(i,j,4), \qquad \forall i,j \in \{1,2,3,4\}.$$

Thus, the 16 grounded clauses are:

$$Val(1,1,1) \vee Val(1,1,2) \vee Val(1,1,3) \vee Val(1,1,4),$$

$$Val(1,2,1) \vee Val(1,2,2) \vee Val(1,2,3) \vee Val(1,2,4),$$

$$Val(1,3,1) \vee Val(1,3,2) \vee Val(1,3,3) \vee Val(1,3,4),$$

$$Val(1,4,1) \vee Val(1,4,2) \vee Val(1,4,3) \vee Val(1,4,4),$$

$$Val(2,1,1) \vee Val(2,1,2) \vee Val(2,1,3) \vee Val(2,1,4),$$

$$Val(2,2,1) \vee Val(2,2,2) \vee Val(2,2,3) \vee Val(2,2,4),$$

$$Val(2,3,1) \vee Val(2,3,2) \vee Val(2,3,3) \vee Val(2,3,4),$$

$$Val(2,4,1) \lor Val(2,4,2) \lor Val(2,4,3) \lor Val(2,4,4),$$

$$Val(3,1,1) \lor Val(3,1,2) \lor Val(3,1,3) \lor Val(3,1,4),$$

$$Val(3,2,1) \lor Val(3,2,2) \lor Val(3,2,3) \lor Val(3,2,4),$$

$$Val(3,3,1) \lor Val(3,3,2) \lor Val(3,3,3) \lor Val(3,3,4),$$

$$Val(3,4,1) \lor Val(3,4,2) \lor Val(3,4,3) \lor Val(3,4,4),$$

$$Val(4,1,1) \lor Val(4,1,2) \lor Val(4,1,3) \lor Val(4,1,4),$$

$$Val(4,2,1) \lor Val(4,2,2) \lor Val(4,2,3) \lor Val(4,2,4),$$

$$Val(4,3,1) \lor Val(4,3,2) \lor Val(4,3,3) \lor Val(4,3,4),$$

$$Val(4,4,1) \lor Val(4,4,2) \lor Val(4,4,3) \lor Val(4,4,4).$$

A2. At most one value per cell

The original axiom is

$$\forall i \forall j \forall v_1 \forall v_2 \Big(Val(i,j,v_1) \land Val(i,j,v_2) \to v_1 = v_2 \Big).$$

Eliminating implication:

$$\forall i \forall j \forall v_1 \forall v_2 \Big(\neg Val(i,j,v_1) \lor \neg Val(i,j,v_2) \lor (v_1 = v_2) \Big).$$

For $v_1 \neq v_2$, the clause becomes:

$$\neg Val(i,j,v_1) \lor \neg Val(i,j,v_2).$$

Hence, for every cell $(i,j) \in \{1,2,3,4\}^2$ and every pair $(a,b) \in \mathcal{V}_2$, we generate the six clauses:

$$\neg Val(i,j,1) \lor \neg Val(i,j,2),$$

$$\neg Val(i,j,1) \lor \neg Val(i,j,3),$$

$$\neg Val(i,j,1) \lor \neg Val(i,j,4),$$

$$\neg Val(i,j,2) \lor \neg Val(i,j,3),$$

$$\neg Val(i,j,2) \lor \neg Val(i,j,4),$$

$$\neg Val(i,j,3) \lor \neg Val(i,j,4).$$

Therefore, A2 contributes $16 \times 6 = 96$ grounded clauses in total.

A3. Row uniqueness

The original axiom is

$$\forall i \forall j_1 \forall j_2 \forall v \, \Big(Val(i, j_1, v) \wedge Val(i, j_2, v) \wedge j_1 \neq j_2 \rightarrow \bot \Big).$$

Eliminating implication:

$$\forall i \forall j_1 \forall j_2 \forall v \, \Big(\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v) \vee (j_1 = j_2) \Big).$$

For $j_1 \neq j_2$, the clausal form is:

$$\neg Val(i, j_1, v) \vee \neg Val(i, j_2, v).$$

Hence, for every row $i \in \{1, 2, 3, 4\}$, every value $v \in \{1, 2, 3, 4\}$, and every pair $(c_1, c_2) \in \mathcal{P}_2$, we generate the six clauses:

$$\neg Val(i, 1, v) \vee \neg Val(i, 2, v),$$

$$\neg Val(i, 1, v) \vee \neg Val(i, 3, v),$$

$$\neg Val(i, 1, v) \vee \neg Val(i, 4, v),$$

$$\neg Val(i, 2, v) \vee \neg Val(i, 3, v),$$

$$\neg Val(i, 2, v) \vee \neg Val(i, 4, v),$$

$$\neg Val(i, 3, v) \vee \neg Val(i, 4, v).$$

Therefore, A3 contributes $4 \times 4 \times 6 = 96$ grounded clauses in total.

A5. Given clues are enforced

The original axiom is

$$\forall i \forall j \forall v \, \Big(Given(i, j, v) \rightarrow Val(i, j, v) \Big).$$

Eliminating implication:

$$\forall i \forall j \forall v \, \Big(\neg Given(i, j, v) \vee Val(i, j, v) \Big).$$

Using the two given facts in this instance, we obtain the two active clauses:

$$\neg Given(3, 3, 2) \vee Val(3, 3, 2),$$

$$\neg Given(4, 1, 3) \vee Val(4, 1, 3).$$

Since $Given(3,3,2)$ and $Given(4,1,3)$ are facts of the KB, we infer:

$$Val(3,3,2), \qquad Val(4,1,3).$$

Combining with A2, we get the remaining exclusions for the two fixed cells:

$$\neg Val(3,3,1), \quad \neg Val(3,3,3), \quad \neg Val(3,3,4),$$

$$\neg Val(4,1,1), \quad \neg Val(4,1,2), \quad \neg Val(4,1,4).$$

A6. Column uniqueness

The original axiom is

$$\forall j \forall i_1 \forall i_2 \forall v \, \Big(Val(i_1,j,v) \wedge Val(i_2,j,v) \wedge i_1 \neq i_2 \rightarrow \bot \Big).$$

Eliminating implication:

$$\forall j \forall i_1 \forall i_2 \forall v \, \Big(\neg Val(i_1,j,v) \vee \neg Val(i_2,j,v) \vee (i_1 = i_2) \Big).$$

For $i_1 \neq i_2$, the clause becomes:

$$\neg Val(i_1,j,v) \vee \neg Val(i_2,j,v).$$

Hence, for every column $j \in \{1,2,3,4\}$, every value $v \in \{1,2,3,4\}$, and every pair $(r_1, r_2) \in \mathcal{P}_2$, we generate the six clauses:

$$\neg Val(1,j,v) \vee \neg Val(2,j,v),$$

$$\neg Val(1,j,v) \vee \neg Val(3,j,v),$$

$$\neg Val(1,j,v) \vee \neg Val(4,j,v),$$

$$\neg Val(2,j,v) \vee \neg Val(3,j,v),$$

$$\neg Val(2,j,v) \vee \neg Val(4,j,v),$$

$$\neg Val(3,j,v) \vee \neg Val(4,j,v).$$

Therefore, A6 contributes $4 \times 4 \times 6 = 96$ grounded clauses in total.

A4. Horizontal less-than constraint

The original axiom is

$$\forall i \forall j \forall v_1 \forall v_2 \, \Big(LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \rightarrow Less(v_1,v_2) \Big).$$

Eliminating implication:

$$\forall i \forall j \forall v_1 \forall v_2 \Big(\neg LessH(i,j) \vee \neg Val(i,j,v_1) \vee \neg Val(i,j+1,v_2) \vee Less(v_1,v_2)\Big).$$

In this instance, the only active fact is

$$LessH(4,1).$$

Thus, we ground all invalid pairs (v_1, v_2) for which $v_1 < v_2$ is false, i.e.

$$\mathcal{I}_< = \{(1,1),(2,1),(2,2),(3,1),(3,2),(3,3),(4,1),(4,2),(4,3),(4,4)\}.$$

Hence, the 10 active clauses are:

$$\neg Val(4,1,1) \vee \neg Val(4,2,1),$$

$$\neg Val(4,1,2) \vee \neg Val(4,2,1),$$

$$\neg Val(4,1,2) \vee \neg Val(4,2,2),$$

$$\neg Val(4,1,3) \vee \neg Val(4,2,1),$$

$$\neg Val(4,1,3) \vee \neg Val(4,2,2),$$

$$\neg Val(4,1,3) \vee \neg Val(4,2,3),$$

$$\neg Val(4,1,4) \vee \neg Val(4,2,1),$$

$$\neg Val(4,1,4) \vee \neg Val(4,2,2),$$

$$\neg Val(4,1,4) \vee \neg Val(4,2,3),$$

$$\neg Val(4,1,4) \vee \neg Val(4,2,4).$$

Since we already know $Val(4,1,3)$ from A5, we can further infer:

$$\neg Val(4,2,1), \quad \neg Val(4,2,2), \quad \neg Val(4,2,3).$$

A7. Vertical less-than constraint

The original axiom is

$$\forall i \forall j \forall v_1 \forall v_2 \Big(LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2) \rightarrow Less(v_1,v_2)\Big).$$

Eliminating implication:

$$\forall i \forall j \forall v_1 \forall v_2 \Big(\neg LessV(i,j) \vee \neg Val(i,j,v_1) \vee \neg Val(i+1,j,v_2) \vee Less(v_1,v_2)\Big).$$

There is no active fact of the form $LessV(i, j)$ in this instance, so A7 contributes 0 active grounded clauses.

A8. Horizontal greater-than constraint
The original axiom is

$$\forall i \forall j \forall v_1 \forall v_2 \Big(GreaterH(i, j) \wedge Val(i, j, v_1) \wedge Val(i, j+1, v_2) \rightarrow Less(v_2, v_1)\Big).$$

Eliminating implication:

$$\forall i \forall j \forall v_1 \forall v_2 \Big(\neg GreaterH(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i, j+1, v_2) \vee Less(v_2, v_1)\Big).$$

There is no active fact of the form $GreaterH(i, j)$ in this instance, so A8 contributes 0 active grounded clauses.

A9. Vertical greater-than constraint
The original axiom is

$$\forall i \forall j \forall v_1 \forall v_2 \Big(GreaterV(i, j) \wedge Val(i, j, v_1) \wedge Val(i+1, j, v_2) \rightarrow Less(v_2, v_1)\Big).$$

Eliminating implication:

$$\forall i \forall j \forall v_1 \forall v_2 \Big(\neg GreaterV(i, j) \vee \neg Val(i, j, v_1) \vee \neg Val(i+1, j, v_2) \vee Less(v_2, v_1)\Big).$$

The active facts are:

$$GreaterV(1, 2), \qquad GreaterV(2, 1).$$

For a greater-than constraint, the invalid assignments are those with $v_1 \leq v_2$, i.e.

$$\mathcal{I}_{>} = \{(1,1), (1,2), (1,3), (1,4), (2,2), (2,3), (2,4), (3,3), (3,4), (4,4)\}.$$

For $GreaterV(1, 2)$, the 10 active clauses are:

$$\neg Val(1, 2, 1) \vee \neg Val(2, 2, 1),$$

$$\neg Val(1, 2, 1) \vee \neg Val(2, 2, 2),$$

$$\neg Val(1, 2, 1) \vee \neg Val(2, 2, 3),$$

$$\neg Val(1, 2, 1) \vee \neg Val(2, 2, 4),$$

$$\neg Val(1, 2, 2) \vee \neg Val(2, 2, 2),$$

$$\neg Val(1,2,2) \vee \neg Val(2,2,3),$$

$$\neg Val(1,2,2) \vee \neg Val(2,2,4),$$

$$\neg Val(1,2,3) \vee \neg Val(2,2,3),$$

$$\neg Val(1,2,3) \vee \neg Val(2,2,4),$$

$$\neg Val(1,2,4) \vee \neg Val(2,2,4).$$

For $GreaterV(2,1)$, the 10 active clauses are:

$$\neg Val(2,1,1) \vee \neg Val(3,1,1),$$

$$\neg Val(2,1,1) \vee \neg Val(3,1,2),$$

$$\neg Val(2,1,1) \vee \neg Val(3,1,3),$$

$$\neg Val(2,1,1) \vee \neg Val(3,1,4),$$

$$\neg Val(2,1,2) \vee \neg Val(3,1,2),$$

$$\neg Val(2,1,2) \vee \neg Val(3,1,3),$$

$$\neg Val(2,1,2) \vee \neg Val(3,1,4),$$

$$\neg Val(2,1,3) \vee \neg Val(3,1,3),$$

$$\neg Val(2,1,3) \vee \neg Val(3,1,4),$$

$$\neg Val(2,1,4) \vee \neg Val(3,1,4).$$

A10. Domain constraint
The original axiom is

$$\forall i \forall j \forall v \, \Big(Val(i,j,v) \rightarrow (1 \leq v \leq 4) \Big).$$

In the grounded formulation, this is enforced implicitly by restricting the value domain to $\{1,2,3,4\}$. Therefore, A10 contributes no additional active CNF clause.

Summary For this 4×4 instance:

- A1 contributes 16 clauses,
- A2 contributes 96 clauses,
- A3 contributes 96 clauses,
- A5 contributes 2 active clue clauses,

- A6 contributes 96 clauses,
- A4 contributes 10 active clauses from $LessH(4, 1)$,
- A7 contributes 0 active clauses,
- A8 contributes 0 active clauses,
- A9 contributes 20 active clauses from $GreaterV(1, 2)$ and $GreaterV(2, 1)$.

Hence, the grounded CNF for the selected instance is completely determined by the two given facts, one horizontal less-than fact, two vertical greater-than facts, and the general uniqueness axioms.

2.2 Knowledge base implementation

2.2.1 KB Architecture

Puzzle Instance $\longrightarrow$ `build_fol_facts`, `build_fol_rules` $\longrightarrow$ Facts, Rules

Facts, Rules $\longrightarrow$ `ground_to_clauses` $\longrightarrow$ Clauses

The `KnowledgeBase` consists of:

- **facts**: used by forward and backward chaining (FC/BC)
- **fol_rules**: used by forward and backward chaining (FC/BC)
- **clauses**: used by resolution and SAT-based methods

2.2.2 FOL builder

```
def build_fol_facts(puzzle):
    for Initial_val from puzzle.grid:
        facts.append(Val(r, c, val))

    for puzzle.constraint
        facts.append(LessH(r, c))
        facts.append(GreaterH(r, c))
        facts.append(LessV(r, c))
        facts.append(GreaterV(r, c))

    return facts
```

While the formal FOL specification defines 10 axioms (A1-A10), our inference engine operates on 7 symbolic rules derived from these axioms.

```python
def build_fol_rules(puzzle):
rules = [
# A2:
"name": "cell_uniqueness",
"premises": [Val("?i","?j","?v1"), Val("?i","?j","?v2")],
"condition": lambda b: b["?v1"] != b["?v2"],
"conclusion": ("FALSE",)

# A3:
"name": "row_uniqueness",
"premises": [Val("?i","?j1","?v"), Val("?i","?j2","?v")],
"condition": lambda b: b["?j1"] != b["?j2"],
"conclusion": ("FALSE",)

# A4:
"name": "less_horizontal",
"premises": [LessH("?i","?j"), Val("?i","?j","?v1"), Val("?i","?j1","?v2")],
"condition": lambda b: b["?j1"]==b["?j"]+1 and b["?v1"]>=["?v2"],
"conclusion": ("FALSE",)

# A6:
"name": "col_uniqueness",
"premises": [Val("?i1","?j","?v"), Val("?i2","?j","?v")],
"condition": lambda b: b["?i1"] != b["?i2"],
"conclusion": ("FALSE",)

# A7:
"name": "less_vertical",
"premises": [LessV("?i","?j"), Val("?i","?j","?v1"), Val("?i1","?j","?v2")],
"condition": lambda b: b["?i1"] == b["?i"] + 1 and b["?v1"] >= b["?v2"],
"conclusion": ("FALSE",)

# A8:
"name": "greater_horizontal",
"premises": [GreaterH("?i","?j"), Val("?i","?j","?v1"), Val("?i","?j1","?v2")],
"condition": lambda b: b["?j1"] == b["?j"] + 1 and b["?v1"] <= b["?v2"],
"conclusion": ("FALSE",)

# A9:
"name": "greater_vertical",
"premises": [GreaterV("?i","?j"), Val("?i","?j","?v1"), Val("?i1","?j","?v2")],
"condition": lambda b: b["?i1"] == b["?i"] + 1 and b["?v1"] <= b["?v2"],
"conclusion": ("FALSE",)
]

    return rules
```

The following axioms are intentionally not included in the knowledge base

- **A1 (At least one value per cell)**: This is not modeled as a rule. If a cell does not have any assigned value, it indicates that the search algorithm has not yet assigned one or that no valid value exists for that cell. This situation is naturally handled during the search process rather than through logical inference.
- **A5 (Given values from the start)**: This rule would only be triggered once to introduce initial facts. Instead, this behavior is directly implemented in the `build_fol_facts` function, where `Val(r,c,v)` facts are added at initialization. Therefore, an explicit rule is unnecessary.
- **A10 (Domain constraints)**: Domain restrictions are implicitly enforced by the search procedure, which only generates candidate values within the valid range $\{1, \ldots, N\}$. As a result, invalid values are never introduced into the fact base, making this axiom redundant.

2.2.3 Why All rules conclude FALSE

Problems:

In Design of FOL inference Rules, there is no problem to design all rules conclude FALSE. But designing by this way breaks Forward / Backward chaining logic.

- Forward chaining works: Premises satisfied ⇒ derive new facts
- Backward chaining works: Trying to prove from goal (conclusion), but no rules that conclude Val(...)

A Natural Alternative solution:

A natural first instinct when designing FOL rules for Forward/ Backward chaining is to encode rules that derive new facts. For example:

- If N - 1 cells in the same row/ column are assigned values v1, v2,...,v_(N - 1) ⇒ The remaining cell must be assigned by the only remaining value v_N.
- If N - 1 possible value of a cell is eliminated, Assign the only left value to that cell.
- Eliminate domain values from constraint cells.

$$\forall i, j, v_1 < v_2 \left(LessH(i, j) \ \wedge \ Val(i, j, v_1) \ \rightarrow \ \neg Val(i, j + 1, v_2) \right)$$

$$\forall j \forall i \forall v \ \Big(Val(i,j,v) \rightarrow \neg Val(i,0,v) \vee ... \vee \neg Val(i,j-1,v) \vee \neg Val(i,j+1,v) \vee ... \vee \neg Val(i,N-1,v) \Big)$$

$$\forall j \forall i \forall v \ \Big(Val(i,j,v) \rightarrow \neg Val(0,j,v) \vee ... \vee \neg Val(i-1,j,v) \vee \neg Val(i+1,j,v) \vee ... \vee \neg Val(N-1,j,v) \Big)$$

$$\forall i,j,v \ \Big(\ \Big(\forall u \in \{1,\ldots,N\}, \ (u \neq v \rightarrow \neg Val(i,j,u)) \Big) \ \rightarrow \ Val(i,j,v) \ \Big)$$

$$\forall i,j,v \ \Big(\Big(\forall j' \in \{1,\ldots,N\} \backslash \{j\}, \ \exists v' \in \{1,\ldots,N\}, \ ((v' \neq v) \wedge Val(i,j',v')) \Big) \ \rightarrow \ Val(i,j,v) \Big)$$

$$\forall i,j,v \ \Big(\Big(\forall i' \in \{1,\ldots,N\} \backslash \{i\}, \ \exists v' \in \{1,\ldots,N\}, \ ((v' \neq v) \wedge Val(i',j,v')) \Big) \ \rightarrow \ Val(i,j,v) \Big)$$

These rules would derive new **Val** or t=**NotVal** facts directly. However, this approach faces fundamental problems in the context of Futoshiki.

- To trigger a positive facts **Val**, the inference engine must know all other values of the massive conditon in this case. Which is costly and terrible for the binding and unification in inference engines.
- **NotVal** explodes the fact base, whenever assign **Val**, it eliminates values for all other constraint cells and add **NotVal** as new facts to Knowledge base. Which make KB triggers re-scanning of all rules via unification.

Our design decision

Given the the above limitations, we adopt the integrity constraint formulation: all rules conclude FALSE. How we handle these rules to match with Forward/ Backward chaining is discussed later in this report.

2.2.4 Why symbolic FOL (facts, rules) + separate grounding

We maintain symbolic FOL and their grounded instances separately because of following reasons:

- Symbolic rules with only 7 rules are used for Forward chaining and Backward chaining engines via unification which is true FOL inference.

- Grounded instances(clauses) are used by SAT valid solution checker and resolution which require propositional CNF.
- Storing them both separately to avoids re-grounding at inference time and keep the FOL structure explicit.

Chapter 3

Unification and Binding Mechanism

3.1 Core idea

As discuss earlier, our design of FOL rules only derive **FALSE**. So to detect contradictions, the system applied unification engine. Each rule consists of premises, condition and a conclusion (FALSE). The system attempts to find a consistent variable assignment (binding) from facts that satisfies all premises. Binding successes mean contradiction found.

Example: unifying a pattern `("Val", "?i", "?j1", "?v")` with a specific ground fact `("Val", 0, 1, 5)` gives the substitution

$$\{?i : 0,\ ?j1 : 1,\ ?v : 5\}.$$

3.2 Process overview

- Select a rules with conclusion **False**
- Try to unify all its premises with known facts.
- If a consistent binding exists ⇒ contradiction found
- If fails, loop back try other rule
- If all rules fail ⇒ No contradiction found

3.3 Function envolved

3.3.1 _apply_bindings(pattern, bindings)

Purpose: Performing grounding by substituting variables in `pattern` (a premise) with `bindings` to reduce the search space.

The function iterates through each element p of the pattern. If p is a variable, that is, a string starting with `?`, and it exists in the bindings, it is replaced by the bound value. Otherwise, if p is a constant such as `"Val"`, `"LessH"`, or an integer, it remains unchanged.

The function returns a substituted tuple that allows the algorithm to focus on constraints relevant to the current assignment.

```
function _apply_bindings(pattern, bindings):
    return tuple where:
        each variable is replaced if bound,
        otherwise remains unchanged
```

3.3.2 __unify_one(pattern, fact, bindings)

Purpose: Matching a pattern (a binded premise) against a ground fact to determine a consistent variable assignment.

First, the function compares the lengths of the pattern and the fact. If they are different, it immediately returns `None`.

It then creates a copy `b` of the current bindings to test new assignments without affecting other branches of the inference search.

```
function _unify_one(pattern, fact, bindings):
    if length(pattern) != length(fact):
        return None

    new_bindings = copy(bindings)

    for each (p, f) in (pattern, fact):
        if p is a variable:
            if p already in bindings:
                if bindings[p] != f:
                    return None
            else:
                bind p -> f
        else:
            if p != f:
                return None

    return new_bindings
```

3.3.3 __unify_all(premises, facts, bindings)

Purpose: executing a recursive search to resolve a list of premises.

The function ensures that variables in a FOL rule receive consistent values throughout the process by combining `_unify_all` and `_apply_bindings`.

It iterates through all available facts in the KB to ensure that no contradiction is overlooked. `_unify_all` uses `_apply_bindings` to update identified variables inside premises and maintain consistency during the process. `_unify_one` is used as a checker to determine whether the pattern matches the fact.

Using a generator (`yield`) allows the algorithm to return a valid binding set immediately upon discovery without traversing the entire search space first, which optimizes memory usage.

The function iterates sequentially through the elements p from the pattern and f from the fact.

- If p is a variable, the function verifies consistency with existing bindings or registers a new binding if p is unbound.
- If p is a constant, the function checks for an exact match between p and f.

The function returns the updated bindings if all elements match successfully; otherwise, it returns `None` to indicate that the pattern is incompatible with the fact.

```
function _unify_all(premises, facts, bindings):
    if premises is empty:
        yield bindings
        return

    first = premises[0]
    rest  = premises[1:]

    pattern = APPLY_BINDINGS(first, bindings)

    for each fact in facts:
        new_bindings = _unify_one(pattern, fact, bindings)
        if new_bindings is not None:
            yield from unify_all(rest, facts, new_bindings)
```

3.3.4 __derive__false(rules, facts)

Traverse through all rules, apply __unify__all for all rules whose conclusion is False.

```
function _derive_false(rules, facts):
    for each rule in rules:
        if rule.conclusion != FALSE:
            continue

```

```
        for each binding in _unify_all(rule.premises, facts, {}):
            if rule.condition(binding) is true:
                return True   // contradiction found

    return False
```

3.4 Limitations

1. The `_unify_all` process needs to traverse the entire KB for every premise of a rule. This leads to significant cost as the number of facts increases.

2. The execution time is slow because `derive_false` is invoked independently at each assignment step. This causes the algorithm to reprove previously proved subgoals, resulting in inefficient computational waste.

3. The algorithm uses a deep recursive structure, so it needs copies of bindings and facts to ensure data isolation. This significantly increases search tree depth and leads to high peak memory consumption.

Chapter 4

Forward Chaining

4.1 Overview.

The implemented algorithm performs First Order Logic (FOL) Forward chaining using *Modus Ponens* combines with constraint propagation over domains.

4.2 Key Design: Domain-Based Elimination.

Problem: All constraint rules in the system derive `FALSE` rather than new positive facts. As a result, a naive forward chaining approach would only detect contradictions and would not be able to derive new `Val` facts.

To address this limitation, we adopt a domain-based elimination strategy:

- For each unassigned cell (i, j) and each candidate value v, we temporarily assume `Val(i,j,v)`.
- We then check whether any rule derives `FALSE` via unification.
- If a contradiction is detected, v is eliminated from the domain of (i, j).
- If no early contradiction found. Progressively reduce domain of all variable (using check rule derive False via unification) until either:
  - A complete assigment is reach (no more reduction in domains across all variable can be done): No contradiction found
  - A contradiction occurs
- If a domain of cell reduces to 0, return none solution found
- When the domain of a cell reduces to a single value, `Val(i,j,v)` is derived as a new fact via Modus Ponens.

- Return full/ partial solution solution if reaching a complete state. Otherwise, loop back.

```
function forward_chaining_solve(puzzle, facts):

    # Step 1: initialize domains
    for each cell (i,j):
        domain(i,j) = {1..N}

    # Step 2: fix domains from known values
    for each Val(i,j,v) in facts:
        domain(i,j) = {v}

    # Step 3: propagate using forward chaining
    apply forward_chaining to prune domains

    # Step 4: check result
    if any domain(i,j) is empty:
        return INVALID

    if all domains size == 1:
        return COMPLETE

    return PARTIAL
```

```
function forward_chaining(facts, rules, domains):
    changed = true

    # Initial contradiction check
    if facts already derive FALSE:
        return INVALID, NOT_COMPLETE

while changed:
    changed = false

    # Domain elimination (Modus Ponens + propagation)
    for each cell (i,j) with |domain(i,j)| > 1:

        to_remove = empty set

        for each value v in domain(i,j):

            candidate = Val(i,j,v)

            # Tentative assignment
            test_facts = facts + {candidate}
```

```

            # (1) Direct contradiction check (Modus Ponens)
            if any rule derives FALSE from test_facts:
                add v to to_remove
                continue

            # (2) Propagation check (lookahead)
            temp_domains = copy(domains)
            temp_domains(i,j) = {v}

            if propagate_assignment(test_facts, temp_domains) == INVALID:
                add v to to_remove

        # Remove all invalid values
        if to_remove is not empty:
            domain(i,j) = domain(i,j) - to_remove
            changed = true


    # Domain collapse and fact derivation
    for each cell (i,j):

        if domain(i,j) is empty:
            return INVALID, NOT_COMPLETE

        if |domain(i,j)| == 1:
            v = the only value in domain(i,j)
            if Val(i,j,v) not in facts:
                add Val(i,j,v) to facts
                update facts_index
                changed = true


# Final contradiction check
if facts derive FALSE:
    return INVALID, NOT_COMPLETE


# Check completeness
if all domains have size 1:
    return VALID, COMPLETE
else:
    return VALID, PARTIAL
```

```
function propagate_assignment(facts, index, domains):

```

```
changed = true

while changed:
    changed = false

    # Eliminate inconsistent values
    for each cell (i,j) with |domain(i,j)| > 1:
        for each value w in domain(i,j):
            if adding Val(i,j,w) derives FALSE:
                remove w from domain(i,j)
                changed = true

    # Domain collapse handling
    for each cell (i,j):

        if domain(i,j) is empty:
            return INVALID

        if |domain(i,j)| == 1:
            w = only value
            if Val(i,j,w) not in facts:
                add Val(i,j,w) to facts
                update index
                changed = true

return VALID
```

This approach is similar to explicitly encoding elimination rules.

4.3 Limitations of Forward Chaining.

Forward chaining alone is insufficient to solve all puzzles. Since it derives facts only from initial existing given values. If there are no varible (cell) has the length domain shrinking to one, the algorithm get stuck.

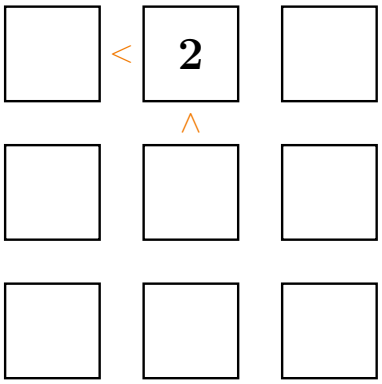
Therefore, forward chaining is combined with backtracking, where it serves as an effective pruning technique to reduce the search space.

```
function HYBRID_SOLVE(puzzle, facts, rules):

    # Step 1: initialize domains
    for each cell (i,j):
        domain(i,j) = {1..N}

    for each Val(i,j,v) in facts:
        domain(i,j) = {v}
```

```

    return BACKTRACK(facts, domains)


function BACKTRACK(facts, domains):

    # Step 2: propagate constraints (FC)
    apply forward_chaining(facts, domains)

    if any domain is empty:
        return FAILURE

    if all domains size == 1:
        return SOLUTION

    # Step 3: choose variable (MRV)
    select cell (i,j) with smallest domain > 1

    # Step 4: try values
    for each v in domain(i,j):
        new_domains = copy(domains)
        new_domains(i,j) = {v}

        new_facts = facts + {Val(i,j,v)}

        result = BACKTRACK(new_facts, new_domains)
        if result != FAILURE:
            return result

    return FAILURE
```

4.4 Illustrative Example

To show how the forward chaining solver runs in practice, consider the following 3×3 Futoshiki puzzle. Every empty cell initially has domain $\{1, 2, 3\}$.



1. Initial Knowledge Base and Domains

The initial knowledge base contains only the given value and the inequality constraints:

$$KB_0 = \{Val(1,2,2),\ LessH(1,1),\ LessV(1,2)\}.$$

The solver uses the following contradiction rules:

$$Val(i,j,v) \wedge Val(i,k,v) \rightarrow FALSE \qquad (j \neq k)$$

$$Val(i,j,v) \wedge Val(k,j,v) \rightarrow FALSE \qquad (i \neq k)$$

$$LessH(i,j) \wedge Val(i,j,v_1) \wedge Val(i,j+1,v_2) \wedge (v_1 \geq v_2) \rightarrow FALSE$$

$$LessV(i,j) \wedge Val(i,j,v_1) \wedge Val(i+1,j,v_2) \wedge (v_1 \geq v_2) \rightarrow FALSE$$

Initial domains:

$$\begin{array}{lll} D(x_{11}) = \{1,2,3\}, & D(x_{12}) = \{2\}, & D(x_{13}) = \{1,2,3\}, \\ D(x_{21}) = \{1,2,3\}, & D(x_{22}) = \{1,2,3\}, & D(x_{23}) = \{1,2,3\}, \\ D(x_{31}) = \{1,2,3\}, & D(x_{32}) = \{1,2,3\}, & D(x_{33}) = \{1,2,3\}. \end{array}$$

2. Apply Forward chaining

Step 1: Process x_{11}.
Current domain:

$$D(x_{11}) = \{1,2,3\}.$$

Test candidate $x_{11} = 1$:

$$test_KB \cup \{Val(1,1,1)\}.$$

No rule derive False $\Rightarrow$ pass **direct contradiction check**

Temporarily assign domain of x_{11}

$$temp_D(x_{11}) = \{1\}.$$

Then continue to **propagation check** for whole table
No rule derive False $\Rightarrow$ pass **propagation check**
So we remmain value **1** in $D(x_{11})$
Test candidate $x_{11} = 2$:

$$test_KB \cup \{Val(1, 1, 2)\}.$$

Now both $Val(1, 1, 2)$ and $Val(1, 2, 2)$ are true in the same row, so the row uniqueness rule derives $FALSE$. Therefore,

$$2 \notin D(x_{11}).$$

Test candidate $x_{11} = 3$:

$$test_KB \cup \{Val(1, 1, 3)\}.$$

Since $LessH(1, 1)$ means $x_{11} < x_{12}$ and $Val(1, 2, 2)$ is already known, the choice $x_{11} = 3$ implies $3 \not< 2$, so the horizontal inequality rule derives $FALSE$. Therefore,

$$3 \notin D(x_{11}).$$

Hence

$$D(x_{11}) = \{1\}.$$

Because the domain collapses to a singleton, we infer

$$Val(1, 1, 1).$$

$$KB_0 \cup \{Val(1, 1, 1)\}.$$

Step 2: Process x_{12}.
Current domain:

$$D(x_{22}) = \{2\}.$$

Already have fixed value, skip this cell.

Step 3: Process x_{13}.
Current domain:

$$D(x_{13}) = \{1, 2, 3\}.$$

Test candidate $x_{13} = 1$:

$$test_KB \cup \{Val(1, 3, 1)\}.$$

Now both $Val(1, 1, 1)$ and $Val(1, 3, 1)$ are true in the same row, so the row uniqueness rule derives $FALSE$. Therefore,

$$1 \notin D(x_{13}).$$

Test candidate $x_{13} = 2$:

$$test_KB \cup \{Val(1, 3, 2)\}.$$

Now both $Val(1, 2, 2)$ and $Val(1, 3, 2)$ are true in the same row, so the row uniqueness rule derives $FALSE$. Therefore,

$$2 \notin D(x_{13}).$$

Test candidate $x_{13} = 3$:

$$test_KB \cup \{Val(1, 3, 3)\}.$$

No rule derive False $\Rightarrow$ pass **direct contradiction check**
Temporarily assign domain of x_{11}

$$temp_D(x_{13}) = \{3\}.$$

Then continue to **propagation check** for whole table
No rule derive False $\Rightarrow$ pass **propagation check**
So we remmain value **3** in $D(x_{13})$
Hence

$$D(x_{13}) = \{3\}.$$

Because the domain collapses to a singleton, we infer

$$Val(1, 3, 3).$$

$$KB_0 \cup \{Val(1,3,3)\}.$$

Step 4: Process x_{21}.
Current domain:

$$D(x_{13}) = \{1,2,3\}.$$

Test candidate $x_{13} = 1$:

$$test_KB \cup \{Val(2,1,1)\}.$$

Now both $Val(2,1,1)$ and $Val(1,1,1)$ are true in the same column, so the column uniqueness rule derives $FALSE$. Therefore,

$$1 \notin D(x_{21}).$$

Test candidate $x_{13} = 2$:

$$test_KB \cup \{Val(2,1,2)\}.$$

It passes **direct contradiction check**.
Temporarily assign domain of x_{21}

$$temp_D(x_{11}) = \{2\}.$$

It passes **propagation check**.
So we remmain value **2** in $D(x_{21})$
Test candidate $x_{21} = 3$:

$$test_KB \cup \{Val(2,1,3)\}.$$

It passes **direct contradiction check**.
Temporarily assign domain of x_{21}

$$temp_D(x_{11}) = \{3\}.$$

Continue with **propagation check**, algorithm skips assigned cells x_{11}, x_{12}, x_{13}, check no derive False in x_{21}.
Continue to cell x_{22} with $D(x_{22}) = \{1,2,3\}$.
Check rules derive False respectively for each value in $temp_D(x_{22}) = \{1,2,3\}$

- $test_KB \cup \{Val(2,2,1)\}$ contradict to $Val(1,2,2)$ and $LessV(1,2)$.

- $test_KB \cup \{Val(2,2,2)\}$ contradict to $Val(1,2,2)$.
- $test_KB \cup \{Val(2,2,3)\}$ contradict to $Val(2,1,3)$.

which make $temp_D(x_{22})$ empty $\Rightarrow$ fails **propagation check**.
Hence

$$3 \notin D(x_{21}).$$

Because the domain collapses to a singleton, we infer

$$Val(2,1,2).$$

$$KB_0 \cup \{Val(2,1,2)\}.$$

Step 5: Process x_{22}.
Current domain:

$$D(x_{22}) = \{1,2,3\}.$$

Test candidate $x_{22} = 1$:

$$test_KB \cup \{Val(2,2,1)\}.$$

It fails **direct contradiction check**, contradict to $Val(1,2,2)$ and $LessV(1,2)$. Therefore,

$$1 \notin D(x_{22}).$$

Test candidate $x_{22} = 2$:
It fails **direct contradiction check**, contradict to $Val(1,2,2)$. Therefore,

$$2 \notin D(x_{22}).$$

Test candidate $x_{22} = 3$:

$$test_KB \cup \{Val(2,2,3)\}.$$

It passes **direct contradiction check**
Temporarily assign domain of x_{22}

$$temp_D(x_{22}) = \{3\}.$$

It passes **propagation check**
So we remmain value **3** in $D(x_{22})$
Hence

$$D(x_{22}) = \{3\}.$$

Because the domain collapses to a singleton, we infer

$$Val(2, 2, 3).$$

$$KB_0 \cup \{Val(2, 2, 3)\}.$$

...

Step 6, 7, 8, 9: Process $x_{23}, x_{31}, x_{32}, x_{33}$.
Similarity, all the invalid value from these domains are reduced by **direct contradiction check**, and their domains collapse to a singletons.

$$KB_0 \cup \{Val(2, 3, 1)\} \cup \{Val(3, 1, 3)\} \cup \{Val(3, 2, 1)\} \cup \{Val(3, 3, 2)\}.$$

At this point every domain is a singleton, so the puzzle is solved.

3. Final Solution

1	$<$ 2	3
2	$\wedge$ 3	1
3	1	**2**

Chapter 5

Backward Chaining

5.1 Algorithm Description

Backward chaining (Backward reasoning) is an *inference* method which works backward from the goal. It use *theorem provers*, *inference engines*, *proof assitants*, and other applications.

5.1.1 SLD Refutation

Instead of directly proving an assignment is correct, the algorithm utilizes a proof via the `derive_false` function to verify that the current state is not yet invalid.

Upon temporarily accepting `Val(i,j,v)`, the algorithm attempts to derive a `FALSE` conclusion from the current KB.

If a `FALSE` conclusion is reached, the assignment is inconsistent and is discarded; otherwise, the assignment is accepted.

5.1.2 Operating Principle

1. **Initialization and Setup**

   Retrieve the list of Facts and First-Order Logic (FOL) Rules from the Knowledge Base (KB). Initialize `assignment` to store the values assigned to the grid.

2. **Variable Selection**

   Select the next unassigned cell using `next_unassigned_cell` by scanning the grid from top-to-bottom and left-to-right.

3. **Goal Formulation**

   The algorithm starts a query to determine whether a cell (i, j) can be assigned a value v, where v is selected from the cell's domain and ranges from 1 to the size of the puzzle.

4. **Consistency Checking and Decision Making**

   - Temporarily assume `Val(i,j,v)` is true and add it to the current fact base.
   - Query: instead of proving `Val(i,j,v)` is correct, the algorithm tries to find any rule that derives `FALSE` (contradiction).
   - It uses unification to match rule variables with facts in the KB.
     - If a contradiction is found, value v is rejected.
     - If no contradiction is found, v is temporarily accepted and stored in the assignment.

5. **Backtracking and Result**

   - If every value in a cell's domain leads to a contradiction, the algorithm backtracks to the previous cell and attempts the next value in its domain.
   - The algorithm succeeds when no empty cells remain on the grid; otherwise, it returns failure if all possible assignments have been explored without finding a valid solution.

5.1.3 Pseudocode

```
function backward_chaining_solve(puzzle, KB):
    # Step 1: Initialization and Setup
    facts  = KB.facts
    rules  = KB.rules
    assign = initial_assignment_from_clues(facts)

    return sld_search(facts, rules, assign)


function sld_search(facts, rules, assignment):
    if assignment is complete:
        return assignment

    # Step 2: Find next unassigned cell
    cell = pick an unassigned cell

    # Step 3: Try each candidate value v
    for each possible value v of cell:
        candidate_facts = facts + Val(cell, v)

        # Step 4: Consistency check via SLD Resolution
        if not derive_false(rules, candidate_facts):
```

```

            # Step 5: Recursive search
            result = sld_search(candidate_facts, rules, assignment + (cell, v))

            if result exists:
                return result

    # Step 6: Backtrack if no value works
    return null


function derive_false(rules, facts):
    check if any rule leading to FALSE can be satisfied
    return true if found, otherwise false
```

5.2 Limitations

1. The `_unify_all` process needs to traverse the entire KB for every premise of a rule. This leads to significant cost as the number of facts increases.

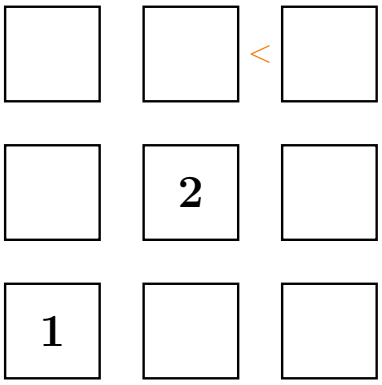
2. The execution time is slow because `derive_false` is invoked independently at each assignment step. This causes the algorithm to reprove previously proved subgoals, resulting in inefficient computational waste.

3. The algorithm uses a deep recursive structure, so it needs copies of bindings and facts to ensure data isolation. This significantly increases search tree depth and leads to high peak memory consumption.

5.3 Illustrative Example

5.3.1 Initial State

The initial puzzle instance is:



The horizontal inequality constraint is:

```
LessH(0,1)
```

The initial knowledge base is defined as follows:

- **Facts:** `{Val(1,1,2), Val(2,0,1), LessH(0,1)}`
- **FOL Rules:** `{cell_uniqueness, row_uniqueness, col_uniqueness, less_horizontal}`
- **Assignment:** `{(1,1): 2, (2,0): 1}`

The algorithm scans cells from top-to-bottom and left-to-right to find the next unassigned cell.

5.3.2 Step-by-Step Execution

Step 1: Assign cell $(0, 0)$

Try $v = 1$**.** The algorithm calls `derive_false` with

$$\texttt{candidate_facts = facts} \cup \texttt{\{Val(0,0,1)\}}.$$

The rule `col_uniqueness` tries to unify `Val(?i1,?j,?v)` and `Val(?i2,?j,?v)` with the condition $?i1 \neq ?i2$.

- The bindings $\{?i1 : 0,\ ?j : 0,\ ?v : 1\}$ match `Val(0,0,1)`.
- The remaining facts are scanned to find another fact `Val(?i2,0,1)` with $?i2 \neq 0$.
- The fact `Val(2,0,1)` is found.
- Since $0 \neq 2$, the rule derives `FALSE`.

Therefore, $v = 1$ is rejected.

Try $v = 2$**.** The algorithm calls `derive_false` with `{Val(0,0,2)}` added to the fact base.

- `col_uniqueness`: scan for another `Val(?i2,0,2)` $\Rightarrow$ none found.
- `row_uniqueness`: scan for another `Val(0,?j2,2)` $\Rightarrow$ none found.
- `cell_uniqueness`: there is no `Val(0,0,?)` in the current facts $\Rightarrow$ none found.
- `less_horizontal`: `LessH(0,1)` exists, but `Val(0,1,?)` is not yet assigned $\Rightarrow$ none found.

Hence, `FALSE` is not derivable, so $v = 2$ is accepted.

The updated assignment becomes:

`{(0,0): 2, (1,1): 2, (2,0): 1}`.

The fact base is updated with `Val(0,0,2)`.

Step 2: Assign cell $(0, 1)$

Try $v = 1$**.** The algorithm checks with

`candidate_facts = facts` $\cup$ `{Val(0,1,1)}`.

- `col_uniqueness`: scan for another `Val(?i2,1,1)` $\Rightarrow$ none found.
- `row_uniqueness`: scan for another `Val(0,?j2,1)` $\Rightarrow$ none found.
- `cell_uniqueness`: there is no `Val(0,1,?)` in the current facts $\Rightarrow$ none found.
- `less_horizontal`: `LessH(0,1)` exists, but `Val(0,2,?)` is not yet assigned $\Rightarrow$ none found.

Therefore, `FALSE` is not derivable, and $v = 1$ is accepted.

The updated assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1}`.

Step 3: Assign cell $(0, 2)$

Try $v = 1$**.**

- `row_uniqueness`: `Val(0,1,1)` is already in the fact base $\Rightarrow$ derives `FALSE`.

Therefore, $v = 1$ is rejected.

Try $v = 2$**.**

- `row_uniqueness`: `Val(0,0,2)` is already in the fact base $\Rightarrow$ derives `FALSE`.

Therefore, $v = 2$ is rejected.

Try $v = 3$**.**

- `less_horizontal`: `LessH(0,1)` and `Val(0,1,1)` and `Val(0,2,3)` does not derive `FALSE`.
- All other checks pass.

Therefore, `FALSE` is not derivable, and $v = 3$ is accepted.

The updated assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1, (0,2):3}`.

Step 4: Assign cell $(1, 0)$

Try $v = 1$ **or** $v = 2$**.**

- `col_uniqueness`: `Val(2,0,1)` and `Val(0,0,2)` is already in the fact base ⇒ derives `FALSE`.

Therefore, these values are rejected.

Try $v = 3$**.**

- All checks pass.

Hence, `FALSE` is not derivable, and $v = 3$ is accepted.

The updated assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1, (0,2):3, (1,0):3}`.

Step 5: Assign cell $(1, 2)$

Try $v = 1$**.**

- All checks pass.

Therefore, `FALSE` is not derivable, and $v = 1$ is accepted.

The updated assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1, (0,2):3, (1,0):3, (1,2):1}`.

Step 6: Assign cell $(2, 1)$

Try $v = 1$**.**

- `row_uniqueness`: `Val(2,0,1)` is already in the fact base ⇒ derives `FALSE`.

Therefore, $v = 1$ is rejected.

Try $v = 2$**.**

- `col_uniqueness`: `Val(1,1,2)` is already in the fact base ⇒ derives `FALSE`.

Therefore, $v = 2$ is rejected.

Try $v = 3$**.**

- All checks pass.

Thus, `FALSE` is not derivable, and $v = 3$ is accepted.

The updated assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1, (0,2):3, (1,0):3, (1,2):1, (2,1):3}`.

Step 7: Assign cell $(2, 2)$

The final unassigned cell is $(2, 2)$.

Try $v = 1$**.**

- `row_uniqueness`: `Val(2,0,1)` is already in the fact base $\Rightarrow$ derives `FALSE`.

Therefore, $v = 1$ is rejected.

Try $v = 2$**.**

- All checks pass.

Therefore, `FALSE` is not derivable, and $v = 2$ is accepted.

The final assignment becomes:

`{(0,0):2, (0,1):1, (1,1):2, (2,0):1, (0,2):3, (1,0):3, (1,2):1, (2,1):3, (2,2):2}`.

5.3.3 Final Solution

The resulting solution grid is:

2	1 <	3
3	**2**	1
1	3	**2**

5.4 Experimental Results

5.4.1 Performance Data

Puzzle Size	Difficulty	Execution Time (s)	Peak Memory (KB)
4×4	Easy	0.4580	218.47
4×4	Medium	0.7310	77.43
4×4	Hard	0.2723	36.98
5×5	Easy	12.3745	69.46
5×5	Medium	7.6311	337.22
5×5	Hard	1.0245	57.54
6×6	Easy	884.0448	390.50
6×6	Medium	211.1379	116.15
6×6	Hard	1958.4896	400.62

Table 5.1: Performance results of backward chaining on Futoshiki instances

5.4.2 Scalability and Time Complexity Analysis

The experimental results reveal a significant exponential growth in execution time as the puzzle size N grows.

For 4×4 instances, all test cases are solved in less than one second. This indicates that backward chaining is highly effective when the search space and the number of facts remain small.

For 5×5 instances, the running time increases to a range of approximately 1 to 12 seconds. At this scale, the algorithm is still usable, but the recursive proof process already begins to introduce a noticeable computational delay.

For 6×6 instances, the execution time increases dramatically. The solver requires about 211 seconds for the medium instance and more than 1958 seconds for the hard instance. This increase confirms that the computational overhead of the unification engine becomes a major bottleneck as the number of facts and rule premises grows.

The bottleneck: This spike is primarily due to the Unification Engine as for every assignment, the algorithm must resolves recursively every premise against the entire Fact Base. When the grid size increases, the computational cost becomes increasingly heavy.

5.4.3 Impact of Constraint Density

A trend observed in 4×4 and 5×5 grids is that "Hard" instances are often solved faster than "Easy" ones for the following reason:

- **Early Refutation:** Higher difficulty levels in Futoshiki often have a higher

density of inequality constraints. This allows the `_derive_false` function to detect contradictions earlier and avoid invalid branches.

In the 6×6 hard case, the time doubled compared to the easy version. This suggests that at larger N, the algorithm may encounter "backtracking traps" where it explores extensive invalid branches before a contradiction is derived.

5.4.4 Memory Management

Despite the deep recursion, peak memory usage consistently remains below 500 KB:

- `_unify_all` is a generator using `yield`. This is highly effective because it produces variables instead of storing an exhaustive list of possibilities in memory.
- Even in the 6×6 hard case, memory peaked at only 400.62 KB, proving that the algorithm's resource footprint is well managed.

Chapter 6

A* Search & Heuristic Analysis

6.1 State Space and Problem Formulation

Each search state is a partial assignment of values to an $N \times N$ grid. We represent a state as (`grid`, `domains`), where `domains`$[(i, j)]$ stores the set of values still feasible for cell (i, j) after constraint propagation.

- **Initial state:** The puzzle as given, with prefilled cells fixed to singleton domains.
- **Goal state:** Every cell has a singleton domain, and all row/column/inequality constraints are satisfied.
- **Successor function:** Select one unassigned cell, assign one value from its domain, then apply forward chaining to propagate implications.
- **Step cost:** 1 per assignment (each node corresponds to one deeper assignment level).

Therefore, the A* evaluation function is

$$f(n) = g(n) + h(n),$$

where $g(n)$ is the number of assignments made so far.

6.2 Heuristic Functions

6.2.1 Remaining Unassigned Cells (h_1)

Definition

A cell is *resolved* when its domain is a singleton $|\, D(i, j) \,| = 1$, and *unresolved* otherwise.

The heuristic h_1 counts how many cells remain unresolved in the puzzle:

$$h_1(s) \;=\; \Big|\{\,(i,j) \mid |D(i,j)| > 1\,\}\Big|$$

Admissibility

Proof: Before the puzzle is complete, each unresolved cell must receive exactly one more assignment. Reaching the goal therefore requires at least one action per unresolved cell. Since h_1 counts exactly these cells, it is a lower bound on the remaining cost.

$$h^*(s) \;\geq\; \Big|\{\,(i,j) \mid |D(i,j)| > 1\,\}\Big| \;=\; h_1(s)$$

h_1 is therefore admissible. ■

Consistency

Proof: Applying action a (assigning a value to one unresolved cell) (i,j) will produces successor s' where that cell becomes resolved. Two outcomes are possible:

- *Only (i,j) is resolved:* $h_1(s) = c(s,a,s') + h_1(s') = 1 + h_1(s')$. The condition holds with equality.
- *Propagation resolves additional cells:* When pruning (FC or AC-3) propagates the assignment and collapses further domains to singletons, so:

$$h_1(s) \leq c(s,a,s') + h_1(s')$$

In both cases $h_1(s) \leq c(s,a,s') + h_1(s')$, so h_1 is consistent. ■

Strength

The heuristic is a simple yet reliable approach for solving Futoshiki within A* framework:

- Guarantees strictly both admissible and consistency property which make it a robust formula.
- Counting unassigned cell has computational cost of $O(N^2)$ which is considerably more efficient than other complex heuristic (such as Inequality Chains).

Limits

Since it provides a much looser lower bound, it causes A* to expand many more nodes than a tighter heuristic. The heuristic h_1 treats every unresolved cell as costing exactly

one step, regardless of how constrained that cell is. Two states with the same number of unresolved cells can differ dramatically in how much work remains.

6.2.2 Inequality Chain Coverage (h_2)

Definition

The heuristic assumes that for any maximal contiguous sequence of cells linked by inequality constraints, at least one assignment must be made before the puzzle is fully solved.

When a horizontal and vertical chain share an unresolved cell, resolving that cell may simultaneously progress both chains. Therefor we design the bipartite graph to subtract these shared resolutions to avoid redundant counting, yielding a tighter lower bound than simply summing all active chains. The heuristic is then:

$$h_2(s) \;=\; |\mathcal{H}^*(s)| + |\mathcal{V}^*(s)| - \mu(s)$$

$\mu(s)$: the maximum number of shared unresolved cells that can be used for simultaneous progress in both horizontal and vertical chains.

Operation

1. **Chain Building:**

   Horizontal & Vertical chains: A tuple of consecutive cells such that every adjacent pair in the tuple is connected through an inequality constraint ($<$ or $>$).

   Assign each cell a chain ID corresponding to the specific chain to which it belongs.

2. **Active Chains:**

   At each node in the A* search tree, the `h_inequality_chains` function filters out unresolved chains.

   - A chain is considered **Active** if it contains at least one cell that has not been assigned ($|domain| > 1$).
   - The goal is to estimate the minimum number of assignment operations required to resolve all identified chains

3. **Bipartite Matching:** This is the core mechanism to maintain **Admissibility** (heuristic must not overcome actual cost).

   - **Graph Construction:** Build a bipartite graph where the left vertex set represents horizontal chains ($active_h$) and the right vertex set represents vertical chains ($active_v$).

- **Edge Definition:** An edge exists between horizontal chain H_i and vertical chain V_j if, and only if, they intersect at an empty cell.
- **Significance:** Assigning a value to the intersecting empty cell allows for the simultaneous "progression" of both the horizontal and vertical chains.
- **Objective:** The goal is to estimate the minimum number of assignment operations required to exhaustively resolve all active chains.

The matching value $\mu(s)$ is computed using a DFS-based augmenting-path algorithm.

```
function maximum_bipartite_matching(adjacency):
    matched_right = {}

    function augment(left, seen):
        for each right in adjacency[left]:
            if right in seen:
                continue
            seen.add(right)

            if right not in matched_right or augment(matched_right[right],
    seen):
                matched_right[right] = left
                return true

        return false

    size = 0
    for each left in adjacency:
        if augment(left, empty_set):
            size = size + 1

    return size

```

Admissibility

Proof: Consider any active chain $C \in \mathcal{H}^*(s) \cup \mathcal{V}^*(s)$. By definition, C contains at least one unresolved cell, so at least one assignment must be made to progress it. Therefore each active chain requires at least one future action, and $|\mathcal{H}^*(s)| + |\mathcal{V}^*(s)|$ is a lower bound on remaining cost before the matching correction.

The maximum matching $\mu(s)$ identifies pairs (C_H, C_V) that share an unresolved cell. A single assignment to that shared cell simultaneously progresses both C_H and C_V, meaning both chains can be advanced by one action rather than two. The matching

correction subtracts exactly these shared actions, so no action is counted fewer times than it actually occurs in any completion of the puzzle. Formally:

$$h^*(s) \;\geq\; |\mathcal{H}^*(s)| + |\mathcal{V}^*(s)| - \mu(s) \;=\; h_2(s)$$

h_2 is therefore admissible. ■

Consistency

Proof: Applying one assignment action a to cell (i, j) produces successor s'. Two effects on active chains are possible:

- *Exactly one chain is deactivated:* (i, j) was the last unresolved cell in chain C. Then $|\mathcal{H}^*(s')| + |\mathcal{V}^*(s')| \leq |\mathcal{H}^*(s)| + |\mathcal{V}^*(s)| - 1$. The matching can decrease by at most 1, so:

$$h_2(s') \geq h_2(s) - 1 - 1 + 0 \;\Rightarrow\; h_2(s) \leq 1 + h_2(s')$$

- *Two linked chains are deactivated simultaneously:* (i, j) was shared by $C_H \in \mathcal{H}^*(s)$ and $C_V \in \mathcal{V}^*(s)$, and it was the last unresolved cell in both. The chain count drops by 2, but the matching also loses the edge (C_H, C_V), so μ decreases by at most 1:

$$h_2(s') \;\geq\; h_2(s) - 2 + 1 \;=\; h_2(s) - 1 \;\Rightarrow\; h_2(s) \leq 1 + h_2(s')$$

In all cases $h_2(s) \leq 1 + h_2(s')$, so h_2 is consistent. ■

Strength

h_2 is strictly stronger than h_1 on any state where inequality chains are present. The reason is h_1 counts unresolved *cells*, while h_2 counts unresolved *chains* after correcting for shared structure. A single long chain spanning k cells contributes k to h_1 but only 1 to h_2 before the matching.

On puzzles with dense inequality constraints (like Futoshiki), h_2 tend to cause A* to expand fewer nodes than h_1, because it discriminates between states that h_1 scores identically.

Limits

The heuristic has two weaknesses. First, the bipartite matching is computed from scratch at every node expansion, which adds $O(|\mathcal{H}^*| \cdot |\mathcal{V}^*|)$ overhead per node. On large

puzzles with many active chains, this cost can offset the reduction in node expansions, making h_2 slower in clock time than h_1 even when it expands fewer nodes.

Second, h_2 is blind to row and column uniqueness constraints. Two cells in the same row that are both unresolved but have no inequality edge between them contribute independently to h_2 with no correction, even though assigning one immediately restricts the other. This means h_2 still underestimates the true cost in non inequality constraint puzzle, motivating the combined heuristic h_3 that takes the maximum of h_1 and h_2.

6.2.3 Combined Heuristic (h_3)

Definition

h_3 takes the maximum of h_1 and h_2 at every state:

$$h_3(s) \;=\; \max\Big(h_1(s),\; h_2(s)\Big)$$

This requires no additional computation beyond evaluating both component heuristics. Chain data from h_2 is built once before the search loop and reused throughout.

h_1 counts unresolved cells and is sensitive to row and column uniqueness pressure, while h_2 counts active inequality chains and is sensitive to ordering constraints. Practically, $h_3 = h_1$ in two situations. The first is early in the search, when most cells are still unresolved and inequality chains are sparse relative to the total cell count. The second is on puzzles with few inequality constraints overall, where $|\mathcal{H}^*| + |\mathcal{V}^*|$ stays small and the chain based estimate never rises above the raw cell count. Conversely, h_2 dominates and $h_3 = h_2$ when the puzzle has dense inequality constraints and propagation has already resolved many cells, leaving a small number of heavily constrained chains.

Admissibility

Proof: Both h_1 and h_2 are admissible, meaning:

$$h_1(s) \leq h^*(s) \qquad \text{and} \qquad h_2(s) \leq h^*(s)$$

$$h_3(s) \;=\; \max(h_1(s),\, h_2(s)) \;\leq\; h^*(s)$$

h_3 is therefore admissible. ■

Strength

By construction, h_3 is at least as informed as both h_1 and h_2 at every node:

$$h_3(s) \geq h_1(s) \qquad \text{and} \qquad h_3(s) \geq h_2(s)$$

This means A* with h_3 never expands a node that h_1 or h_2 alone would have pruned. On puzzles where h_1 and h_2 alternate in dominance across different regions of the search tree which is common on medium to large grids where some rows are inequality dense and others are not. h_3 consistently picks the better estimate at each node without any look ahead or domain copying. The gain over each individual heuristic is therefore most visible on structurally mixed puzzles.

Limits

h_3 inherits the computational cost of h_2: the bipartite matching must still be computed at every node expansion whenever inequality chains are present. On very large puzzles ($N = 9$) with many active chains the benefit of a tighter bound may not fully compensate for the extra time per node compared to using h_1 alone.

A deeper limitation is that it treats h_1 and h_2 independently and picks the larger one, but does not combine their information additively. A heuristic that simultaneously accounts for both unresolved cell count and inequality chain structure in a single coherent estimate could potentially be tighter still. Constructing such a heuristic without sacrificing admissibility or adding computation remains an open challenge within this project.

6.3 Pruning Methods

6.3.1 Forward Chaining Pruning

Forward chaining pruning integrates the FOL inference engine directly into the A* expansion step. Every time A* assigns a value to a cell, the full forward chaining engine runs on the resulting state before that state is ever pushed onto the open list.

Operation

When A* assigns value v to cell (i, j), it adds $\text{Val}(i, j, v)$ to the current fact set and calls `forward_chaining(facts, rules, domains)`, which applies Modus Ponens exhaustively over the grounded FOL rules cell, row, and column uniqueness plus all inequality axioms, then returns a validity flag, a completion flag, an updated fact set, and an updated domain map.

Inside A*, forward chaining is not trying to solve the puzzle. It runs on a partial state whose fact set grows by one assignment per expansion, and its only job is to propagate the consequences of that assignment as far as pure deduction reaches. This is different from the standalone solver, which starts from the full clue set and runs to fixpoint in one pass.

A* uses the four return values directly. A `FALSE` validity flag means some domain collapsed to the empty set so no consistent completion exists and the branch is cut at generation rather than at expansion. A `TRUE` completion flag means every domain is already a singleton and the puzzle is solved without further search. Otherwise, the tightened domains replace the previous ones and the heuristic is recomputed before the successor enters the open list.

```
function prune_fc(facts, rules, domains):
    (is_valid, is_done, new_facts, new_domains) =
        forward_chaining(facts, rules, domains)

    return (is_valid, is_done, new_facts, new_domains)
```

Strength

FC pruning can resolve cells beyond the one just assigned. When a value is fixed, the inequality and uniqueness rules may cascade and force several other cells to singletons in the same propagation pass. This tightens the domain map that the heuristic sees, which in turn produces a more accurate h value and a better node ordering in the open list. On puzzles where given clues or early assignments trigger long inference chains, FC pruning can solve the puzzle entirely during the initial propagation pass before the main search loop even begins.

Limits

FC pruning carries the full cost of a forward chaining pass at every node expansion and at every candidate value tried within that expansion. On a grid of size N, the grounded rule set grows as $O(N^3)$, so each pruning call becomes more expensive as N increases. On large puzzles ($N > 6$) with many candidate values per cell, this overhead accumulates quickly.

6.3.2 AC-3 Pruning

AC-3 enforces arc consistency across the entire domain map after each assignment. Where forward chaining reasons over logical rules, AC-3 reasons directly over value domains: it removes any value from a cell's domain that has no supporting value in a

neighbouring cell, and propagates these removals until no further reduction is possible.

6.3.3 Why not implement FOL in AC-3?

Running the FOL rule engine inside each arc-consistency check would create two nested propagation loops where only one is needed.

The cost of this combination is easiest to see. AC-3 already maintains a queue of arcs and re-examines each arc every time a neighbouring domain shrinks. For an $N \times N$ puzzle, the arc set has $O(N^3)$ entries: $O(N^2)$ arcs from row and column uniqueness per cell pair, plus $O(N)$ inequality arcs per row and column. Each time an arc is dequeued, the support check currently costs $O(N)$ by scanning the neighbour's domain. If a forward chaining pass were substituted for that support check, each dequeue operation would instead trigger a full Modus Ponens fixpoint computation over the grounded rule set, which itself costs $O(N^3)$ per pass in the worst case. The total per node cost would rise from $O(N^4)$ to $O(N^6)$.

There is also a conceptual redundancy. The FOL rules that AC-3 would invoke uniqueness and inequality axioms are exactly the constraints that the arc types `neq`, `lt`, and `gt` already encode directly. Running the FOL engine to derive what AC-3 already checks natively would produce identical domain reductions through a far more expensive path.

Operation

The arc set is built once from the puzzle structure before the search loop starts, in `BUILD_AC3_ARCS`. For every pair of cells (X_i, X_j) that are related by a constraint, two directed arcs are added: one in each direction. Three arc types are used.

An arc of type `neq` between (X_i, X_j) encodes the all different requirement from FOL axioms A3 and A4 (row and column uniqueness). It states that every value in X_i's domain must have at least one value in X_j's domain that differs from it. An arc of type `lt` encodes a less than inequality: every value v in X_i's domain must have at least one value $w > v$ in X_j's domain. An arc of type `gt` encodes a greater than inequality: every value v in X_i's domain must have at least one value $w < v$ in X_j's domain. These three support conditions map directly to the FOL axioms:

$$\begin{aligned} \texttt{neq} &\longleftrightarrow \text{A3 (row uniqueness), A4 (column uniqueness)} \\ \texttt{lt} &\longleftrightarrow \text{A5 (LessH / LessV constraint)} \\ \texttt{gt} &\longleftrightarrow \text{A6 (GreaterH / GreaterV constraint)} \end{aligned}$$

A value $v \in D(X_i)$ is removed when the support check fails, meaning no value in

$D(X_j)$ satisfies the arc condition with v. When $D(X_i)$ shrinks, all arcs that point into X_i are re added to the queue, because their source cells may have lost support.

The arc set is built once and reused at every node rather than rebuilt per node because the constraint structure of the puzzle does not change during search, only the domains change.

A contradiction is detected when any domain empties during propagation. AC-3 returns `FALSE` immediately, and A* discards the successor without adding it to the open list.

```
function build_ac3_arcs(puzzle):
    arcs = []

    for each cell (i, j) in puzzle:
        add row uniqueness arcs of type neq
        add column uniqueness arcs of type neq
        add horizontal inequality arcs of type lt or gt
        add vertical inequality arcs of type lt or gt

    neighbors_of = map each cell to empty list
    for each arc (Xi, Xj, ctype) in arcs:
        neighbors_of[Xj].append((Xi, Xj, ctype))

    return (arcs, neighbors_of)


function prune_ac3(domains, arcs, neighbors_of):
    dom = deep_copy(domains)
    in_queue = set(arcs)
    queue = deque(arcs)

    while queue is not empty:
        (Xi, Xj, ctype) = queue.pop_left()
        remove (Xi, Xj, ctype) from in_queue

        old_size = |dom[Xi]|
        remove every value v from dom[Xi]
            that has no support in dom[Xj] under ctype

        if |dom[Xi]| == old_size:
            continue

        if |dom[Xi]| == 0:
            return (false, false, dom)

        for each arc in neighbors_of[Xi]:
            if arc not in in_queue:
```

```
                queue.push_back(arc)
                add arc to in_queue

    is_done = every domain in dom is singleton
    return (true, is_done, dom)
```

Strength

AC-3 is strictly more powerful than FC as a propagator. It enforces consistency between all pairs of constrained cells simultaneously, not just between the newly assigned cell and its direct neighbours. This means it can detect contradictions earlier and reduce domains further in a single pruning call. On puzzles with dense inequality constraints, AC-3 often collapses several additional cells to singletons per expansion, producing tighter domains for the heuristic.

Limits

The main cost of AC-3 is that it operates on a copy of the full domain map at every node expansion, since domains must not be mutated across branches. Copying an $N \times N$ domain map and running the queue propagation loop adds overhead that grows with both puzzle size and constraint density.

6.4 Core Algorithm Design

At each step A* picks the minimum remaining value state from a priority queue, assigns a value to one cell, propagates constraints, and pushes the resulting successors back onto the queue. Four design decisions determine how this plays out in practice: how states are represented, how the priority queue is ordered, how the next cell to assign is chosen, and how duplicate states are suppressed.

6.4.1 Operation

```
function solve_astar(puzzle, kb, heuristic, pruning):
    initialize domains, facts, rules
    precompute chain_data if heuristic needs inequality chains
    precompute AC-3 arcs if pruning uses AC-3

    apply initial pruning to the start state
    if start state is invalid:
        return failure
    if start state is solved:
        return extracted solution
```

```

    h0 = compute_heuristic(start_domains, heuristic, chain_data)
    open_list = priority queue with (h0, 0, tie, puzzle, facts, domains)
    closed = empty set

    while open_list is not empty:
        (f, g, tie, current_puzzle, current_facts, current_domains) = pop best node

        if current state already appears in closed:
            continue
        add current state to closed

        if every domain is singleton:
            return extracted solution

        cell = select unresolved cell with minimum remaining values
        for each value in domain(cell):
            create successor by assigning value to cell
            apply chosen pruning method

            if successor is invalid:
                continue
            if successor is solved:
                return extracted solution
            if successor is already in closed:
                continue

            new_h = compute_heuristic(successor_domains, heuristic, chain_data)
            if new_h is infinity:
                continue

            push successor into open_list with priority g + 1 + new_h

    return failure
```

State Representation

Each state is stored as a tuple `(f, g, tie, puzzle, facts, domains)`, where `f = g + h` is the priority, `g` is the number of assignments made so far, and `domains` maps every cell (i, j) to the set of values still consistent with the current partial assignment. The `facts` set carries the ground FOL atoms known at this node and is passed to the forward chaining pruner at each expansion.

The `tie` field is an increasing integer counter. The priority queue compares tuple keys from left to right. If two states share the same `f` and `g` values, the queue uses `tie`

as a deterministic tiebreaker before looking at complex state objects. This guarantees a total ordering for queue operations without relying on implementation-specific object comparisons.

Initial Propagation

Before any node is placed on the open list, the algorithm performs a full pass through the initial state with either the forward chaining or AC-3 pruning methods. This pass provides three functions. It can detect if there are empty domains present and if there are then the problem does not have a solution and the search will exit immediately. If it finds that every domain is reduced to a single element then the problem is completely solved by propagation alone and A* will return without expanding a single node. For most cases it will reduce the number of elements for many cells and therefore, the first node placed into the open list reflects all consequences of the known clues and the heuristics (h0) calculated from these reduced domains are closer together than those calculated from the original large domains.

Open List and Node Expansion

Once search begins, successors are pushed as `(g+h, g, tie, puzzle, facts, domains)`. Then algorithm processes nodes based on their f-score in ascending order. When two nodes have the same f-score, it uses g to determine. By prioritizing deeper nodes this way, the search stays focused on reaching the goal rather than wasting time re evaluating shallow states that are unlikely to be the solution.

MRV Variable Selection.

The MRV function operates at a level below the heuristic function during the search. The heuristic (h) determines how the nodes are ordered globally across the search tree while MRV determines how they branch locally inside each individual expansion. An early break occurs after scanning a row when a cell containing exactly two possible values is encountered because there is no smaller domain size; this prevents a full $O(N^2)$ traversal on larger grids when an almost complete cell is discovered.

Successor Generation.

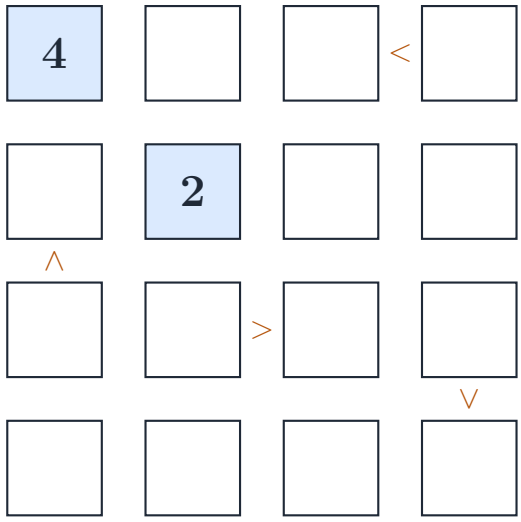
A copy of the current domain map is created for each candidate value in the selected cell's domain. Each copy is set to contain only a single element for that cell and then both the pruning process and calculation of a new heuristic value are performed. Any successors which result in an empty domain are immediately discarded. Successors where all remaining domains have been reduced to single elements are considered solutions and are returned without needing to be added to the open list. The remainder

are inserted with updated `(g+1+h, g+1, tie, ...)` tuples and the tie counter is incremented.

6.4.2 Illustrative Example

We illustrate a hand solve trace on a 4×4 grid implementing Heuristic h_3 (AC-3) combined with AC-3 pruning.

The initial grid is represented as follows:



Constraint Table

Type	Cells	Rule
Pre-filled clue	$(0,0)$	$= 4$
Pre-filled clue	$(1,1)$	$= 2$
Horizontal	$(0,2),\ (0,3)$	$(0,2) < (0,3)$
Horizontal	$(2,1),\ (2,2)$	$(2,1) > (2,2)$
Vertical	$(1,0),\ (2,0)$	$(1,0) < (2,0)$
Vertical	$(2,3),\ (3,3)$	$(2,3) > (3,3)$

Code representation

The procedure `BUILD_AC3_ARCS(puzzle)` reads `puzzle.h_constraints` and `puzzle.v_constraints` and appends arcs of the form `(Xi, Xj, ctype)` where `ctype` $\in$ `{'neq', 'lt', 'gt'}`. For this puzzle:

- `((0,2),(0,3),'lt')` and `((0,3),(0,2),'gt')` from the horizontal $<$;
- `((2,1),(2,2),'gt')` and `((2,2),(2,1),'lt')` from the horizontal $>$;
- `((1,0),(2,0),'lt')` and `((2,0),(1,0),'gt')` from the vertical $<$;

- `((2,3),(3,3),'gt')` and `((3,3),(2,3),'lt')` from the vertical $>$.

1. Initial State (S_0)

The procedure `INITIALIZE_DOMAINS(puzzle)` assigns singleton domains to the two given clues and the full set $\{1, 2, 3, 4\}$ to every empty cell. The current state has $g(S_0) = 0$.

The AC-3 domain computation at S_0 proceeds as follows.

Step 1 (Column and row uniqueness from the clue $(0, 0) = 4$)

Every cell in column 0 and row 0 loses the value 4 via `'neq'` arcs:

$$
\begin{aligned}
D(0,1) &\leftarrow \{1,2,3,4\} \setminus \{4\} = \{1,2,3\} \\
D(0,2) &\leftarrow \{1,2,3\} \\
D(0,3) &\leftarrow \{1,2,3\} \\
D(1,0) &\leftarrow \{1,2,3\} \\
D(2,0) &\leftarrow \{1,2,3\} \\
D(3,0) &\leftarrow \{1,2,3\}
\end{aligned}
$$

Step 2 (Row and column uniqueness from the clue $(1, 1) = 2$) Every other cell in row 1 and column 1 loses the value 2:

$$
\begin{aligned}
D(0,1) &\leftarrow \{1,2,3\} \setminus \{2\} = \{1,3\} \\
D(1,0) &\leftarrow \{1,2,3\} \setminus \{2\} = \{1,3\}
\end{aligned}
$$

Step 3 (Vertical inequality $(1, 0) < (2, 0)$)

Arc `((1,0),(2,0),'lt')`: each $v_x \in D(1, 0) = \{1, 3\}$ needs a support $v_y > v_x$ in $D(2, 0) = \{1, 2, 3\}$. For $v_x = 3$: no $v_y > 3$ exists. Remove 3:

$$D(1,0) \leftarrow \{1\}$$

Arc `((2,0),(1,0),'gt')`: each $v_x \in D(2, 0) = \{1, 2, 3\}$ needs $v_y < v_x$ in $D(1, 0) = \{1\}$. For $v_x = 1$: no $v_y < 1$ exists. Remove 1:

$$D(2,0) \leftarrow \{2,3\}$$

Step 4 (Horizontal inequality $(2, 1) > (2, 2)$)

Arc `((2,1),(2,2),'gt')`: each $v_x \in D(2, 1) = \{1, 3, 4\}$ (after removing 2 in Step 2) needs $v_y < v_x$ in $D(2, 2) = \{1, 2, 3, 4\}$. For $v_x = 1$: no $v_y < 1$ exists. Remove 1:

$$D(2,1) \leftarrow \{3,4\}$$

Arc `((2,2),(2,1),'lt')`: each $v_x \in D(2,2)$ needs $v_y > v_x$ in $D(2,1) = \{3,4\}$. For $v_x = 4$: no $v_y > 4$ exists. Remove 4:

$$D(2,2) \leftarrow \{1,2,3\}$$

Step 5 (Horizontal inequality $(0,2) < (0,3)$**)**

Arc `((0,2),(0,3),'lt')`: each $v_x \in D(0,2) = \{1,2,3\}$ needs $v_y > v_x$ in $D(0,3) = \{1,2,3\}$. For $v_x = 3$: no $v_y > 3$ exists. Remove 3:

$$D(0,2) \leftarrow \{1,2\}$$

Arc `((0,3),(0,2),'gt')`: each $v_x \in D(0,3) = \{1,2,3\}$ needs $v_y < v_x$ in $D(0,2) = \{1,2\}$. For $v_x = 1$: no $v_y < 1$ exists. Remove 1:

$$D(0,3) \leftarrow \{2,3\}$$

Step 6 (Vertical inequality $(2,3) > (3,3)$**)**

Arc `((2,3),(3,3),'gt')`: each $v_x \in D(2,3) = \{1,2,3,4\}$ needs $v_y < v_x$ in $D(3,3) = \{1,2,3,4\}$. For $v_x = 1$: no $v_y < 1$ exists. Remove 1:

$$D(2,3) \leftarrow \{2,3,4\}$$

Arc `((3,3),(2,3),'lt')`: each $v_x \in D(3,3)$ needs $v_y > v_x$ in $D(2,3) = \{2,3,4\}$. For $v_x = 4$: no $v_y > 4$ exists. Remove 4:

$$D(3,3) \leftarrow \{1,2,3\}$$

Remaining iterations

The algorithm continues to dequeue the remaining arcs. Because the propagation from Step 3 forced $D(1,0) = \{1\}$, the `neq` arc `((3,0),(1,0),'neq')` removes 1 from $D(3,0)$, giving $D(3,0) \leftarrow \{2,3\}$. All subsequent arc-revision checks return `FALSE`, and the queue drains without further changes.

Conclusion after `PRUNE_AC3`

No domain is empty, so `IS_VALID = TRUE`. Not all domains are singletons, so `IS_DONE = FALSE` and the A* loop begins.

The reduced domains after the initial pass are:

Cell	Domain	Cell	Domain
$(0,0)$	$\{4\}$	$(2,0)$	$\{2,3\}$
$(0,1)$	$\{1,3\}$	$(2,1)$	$\{3,4\}$
$(0,2)$	$\{1,2\}$	$(2,2)$	$\{1,2,3\}$
$(0,3)$	$\{2,3\}$	$(2,3)$	$\{2,3,4\}$
$(1,0)$	$\{1\}$	$(3,0)$	$\{2,3\}$
$(1,1)$	$\{2\}$	$(3,1)$	$\{1,3,4\}$
$(1,2)$	$\{3,4\}$	$(3,2)$	$\{1,2,3,4\}$
$(1,3)$	$\{3,4\}$	$(3,3)$	$\{1,2,3\}$

Heuristic values at S_0:

$$h_1(S_0) = |\{(i,j) \mid |D(i,j)| > 1\}| = 13$$

From the constraint table, there are two active horizontal chains and two active vertical chains at S_0, and no shared unresolved intersection between an active horizontal chain and an active vertical chain, so $\mu(S_0) = 0$. Therefore:

$$h_2(S_0) = |\mathcal{H}^*(S_0)| + |\mathcal{V}^*(S_0)| - \mu(S_0) = 2 + 2 - 0 = 4$$

Using the combined heuristic definition:

$$h_3(S_0) = \max\Big(h_1(S_0), h_2(S_0)\Big) = \max(13, 4) = 13$$

$g(S_0) = 0 \Rightarrow f(S_0) = 0 + 13 = 13$. The start node is inserted into the open list with priority 13.

2. A* Expansion Loop

Iteration 1 (Expand Node #1)

1. Extract S_0 from the priority queue using an `EXTRACT_MIN` operation. Current $f = 13$, $g = 0$.

2. **Goal check:** several domains have size > 1, so the check fails.

3. **MRV cell selection.** The function scans all cells with $|D| > 1$. The first cells it encounters with the minimum domain size of 2 are $(0,1)$, $(0,2)$, $(0,3)$, $(1,2)$, $(1,3)$, $(2,0)$, $(2,1)$, and $(3,0)$. The MRV scan stops as soon as it encounters domain size 2 (the smallest possible non-singleton size), so MRV selects **cell** $(0,1)$ with $D(0,1) = \{1,3\}$.

4. **Expand: try $v = 1$.** Set $D'(0,1) = \{1\}$ and call `PRUNE_AC3`. The key propagation steps are:

   - Arc `((0,2),(0,1),'neq')`: removes 1 from $D(0,2)$, so $D(0,2) \leftarrow \{2\}$.
   - Arc `((0,3),(0,2),'neq')`: removes 2 from $D(0,3) = \{2,3\}$, so $D(0,3) \leftarrow \{3\}$.
   - Arc `((1,3),(0,3),'neq')`: removes 3 from $D(1,3) = \{3,4\}$, so $D(1,3) \leftarrow \{4\}$.
   - Arc `((1,2),(1,3),'neq')`: removes 4 from $D(1,2) = \{3,4\}$, so $D(1,2) \leftarrow \{3\}$.
   - Further `neq` arcs cascade: $D(2,2) \leftarrow \{1\}$, $D(2,3) \leftarrow \{2\}$, $D(2,0) \leftarrow \{3\}$, $D(3,0) \leftarrow \{2\}$, $D(2,1) \leftarrow \{4\}$, $D(3,1) \leftarrow \{3\}$, $D(3,2) \leftarrow \{4\}$, $D(3,3) \leftarrow \{1\}$.

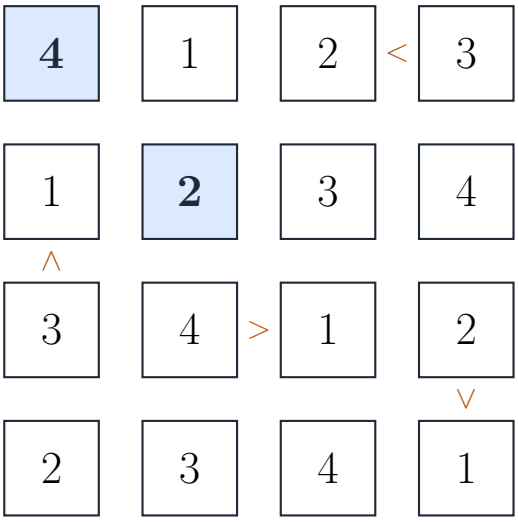
   Every domain collapses to a singleton. `IS_DONE = TRUE`.

5. **Expand: try $v = 3$.** Set $D''(0,1) = \{3\}$ and call `PRUNE_AC3`. The propagation forces $D(0,3) \leftarrow \{2\}$ (row `neq` removes 3), then $D(2,1) \leftarrow \{4\}$ (column `neq` removes 3). With $D(2,1) = \{4\}$, the arc `((2,3),(2,1),'neq')` removes 4 from $D(2,3)$, giving $D(2,3) \leftarrow \{3,4\} \setminus \{4\} \rightarrow \{3\}$. Wait: checking through the full propagation, $D(3,1)$ loses both 3 and 4 in sequence, so $D(3,1) \leftarrow \{1\} \setminus \{1\} = \emptyset$. `PRUNE_AC3` returns `IS_VALID = FALSE`. The branch is discarded; this successor is not inserted into the open list.

Because $v = 1$ triggered `IS_DONE = TRUE`, the algorithm returns the solution immediately without pushing S_1 onto the open list.

3. Goal State

The A* search reaches the goal after **2 node expansions** and **1 search iteration**:



6.5 Experimental Result

This section reports the empirical behavior of the A* variants under the common benchmark setting used throughout the report. The comparison focuses on how heuristic choice and pruning strategy affect runtime, memory usage, and search effort across puzzle sizes and difficulty levels.

6.5.1 Performance Data

Figures 6.1 and 6.2 summarize runtime and peak memory for all six A* variants across grid sizes 4×4, 5×5, and 6×6. Each variant pairs one of three heuristics (h_1: remaining unassigned cells, h_2: inequality chain coverage, h_3: combined maximum) with one of two pruning methods (Forward Chaining or AC-3).

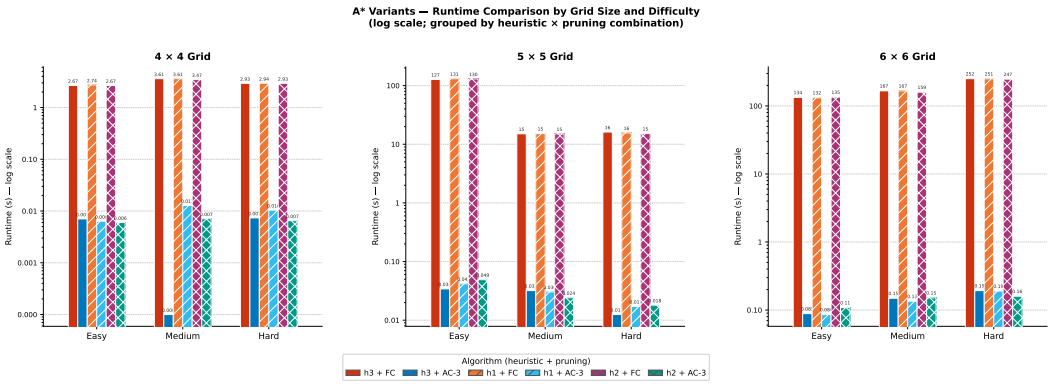


Figure 6.1: Runtime comparison of A* variants across puzzle sizes and difficulties

6.5.2 Pruning Dominates Runtime; Heuristic Choice Is Secondary

The most immediate pattern in Figures 6.1 and 6.2 is that swapping the pruning method changes runtime by two to four orders of magnitude, while swapping the heuristic within the same pruner only lead to a marginal difference.

FC pruning invokes the full FOL unification engine at every node expansion, and the grounded rule set grows as $O(N^3)$, so each pruning call becomes heavier with N. AC-3 operates directly over value domains: once the arc set is built at puzzle-load time, each propagation pass touches only arcs whose neighbour domain has shrunk. The heuristics only reorder nodes in the priority queue; they cannot reduce the cost paid per node. When that per-node cost is large (FC), every ordering performs equally poorly. When it is cheap (AC-3), every ordering finishes quickly and the differences among h_1, h_2, and h_3 stay too small to matter.

6.5.3 FC Runtime Grows with Difficulty, AC-3 Runtime Shrinks

The two pruners respond to difficulty in opposite directions. For the FC variants at 6×6, runtime increases from easy to hard: h_3+FC reads 134 s, 167 s, 252 s. For the AC-3 variants at 5×5, runtime decreases from easy to hard: h_3+AC-3 reads 0.034 s, 0.032 s, 0.012 s. At 6×6 the AC-3 direction reverses slightly (0.089 s $\rightarrow$ 0.15 s $\rightarrow$ 0.19 s), but the spread remains under 110 ms and does not approach the FC range.

The inversion for AC-3 is the same constraint density effect seen in backward chaining: harder puzzles have more inequality arcs, so AC-3 collapses more domains per propagation pass and the search tree stays narrow. FC does not benefit from this because its bottleneck is the unification cost per node, not the number of nodes. More constraints mean more facts in the knowledge base, which makes each `_derive_false` call slower rather than faster.

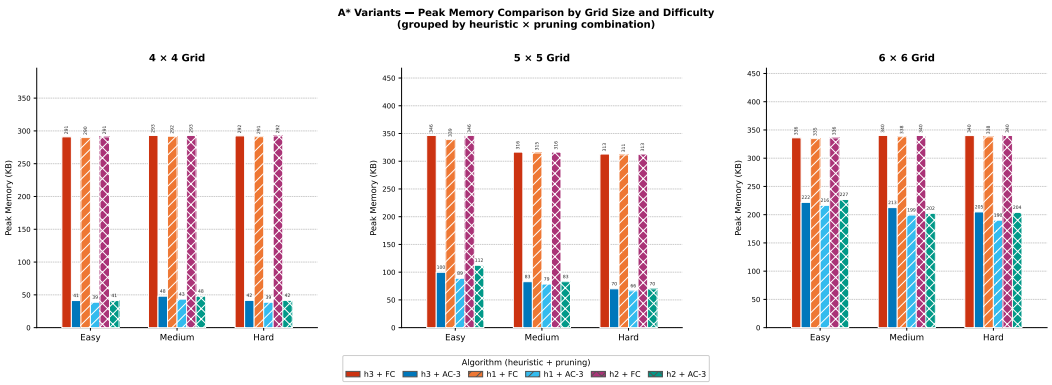


Figure 6.2: Memory comparison of A* variants across puzzle sizes and difficulties

6.5.4 Pruner Determines Memory Tier, Heuristic Is Invisible at Small Sizes

The memory plots show the same two tier split as the runtime plots. All FC variants cluster at 290–346 KB regardless of heuristic, difficulty, or grid size through 6×6. All AC-3 variants sit between 39 and 227 KB depending on grid size and difficulty far below the FC best.

FC variants allocate the full grounded fact base and domain copies at initialisation and hold them for the entire run. That cost depends only on N, so difficulty cannot move it. AC-3 stores domain map copies per open-list node, so its peak memory tracks the widest the search frontier gets.

The heuristic has no visible effect on memory within the FC group, and only a small effect within the AC-3 group. Because the search is so short that matching overhead never accumulates.

Chapter 7

Comparison & Analysis

7.1 Experimental Setup

In this section, we compare all of the algorithms' performance through various puzzle sizes and difficulties. Under a consistent experimental protocol, an unbiased and balanced evaluation can be conducted thereby provide a comprehensive understanding of each algorithm behavior and limitation. For comparative consistency, we **deliberately selected** puzzles that Forward Chaining can solve without getting stuck. Three metrics drive every comparison:

- Runtime (clock seconds).
- Peak memory (KB).
- Inference effort (number of derive false calls or node expansions).

these were chosen because they capture different dimensions of scalability and resource consumption.

7.1.1 Test Suite

All experiments share the same 10 input files spanning 4 grid sizes (4×4 through 7×7) and three difficulty tiers. For grid sizes from 4×4 to 6×6, one testcase is used for each difficulty level (easy, medium, and hard). For 7×7, only one easy testcase is included. The testcase IDs follow the directory in the input folder, in this setup we use `input-01`.

Table 7.1: Test suite used in the experiments

Testcase ID	**Grid Size**	**Difficulty**
Inputs/easy/input_01.txt	4×4	Easy
Inputs/medium/input_01.txt	4×4	Medium
Inputs/hard/input_01.txt	4×4	Hard
Inputs/easy/input_01.txt	5×5	Easy
Inputs/medium/input_01.txt	5×5	Medium
Inputs/hard/input_01.txt	5×5	Hard
Inputs/easy/input_01.txt	6×6	Easy
Inputs/medium/input_01.txt	6×6	Medium
Inputs/hard/input_01.txt	6×6	Hard
Inputs/easy/input_01.txt	7×7	Easy

Hardware and software environment are held constant (Python 3.11, single core timing, same devices), ensuring that observed differences reflect algorithmic behavior rather than system variance.

7.2 Runtime Comparison

7.2.1 Small Grid Scale

To evaluate the scalability of the different Futoshiki solvers, a benchmark was conducted using a log scale line chart. For each algorithm and size pair, the average `time_sec` across all difficulties (easy, medium, and hard) was used. Any 0.0 values were replaced with 10^{-4} solely to avoid undefined values on the logarithmic scale.

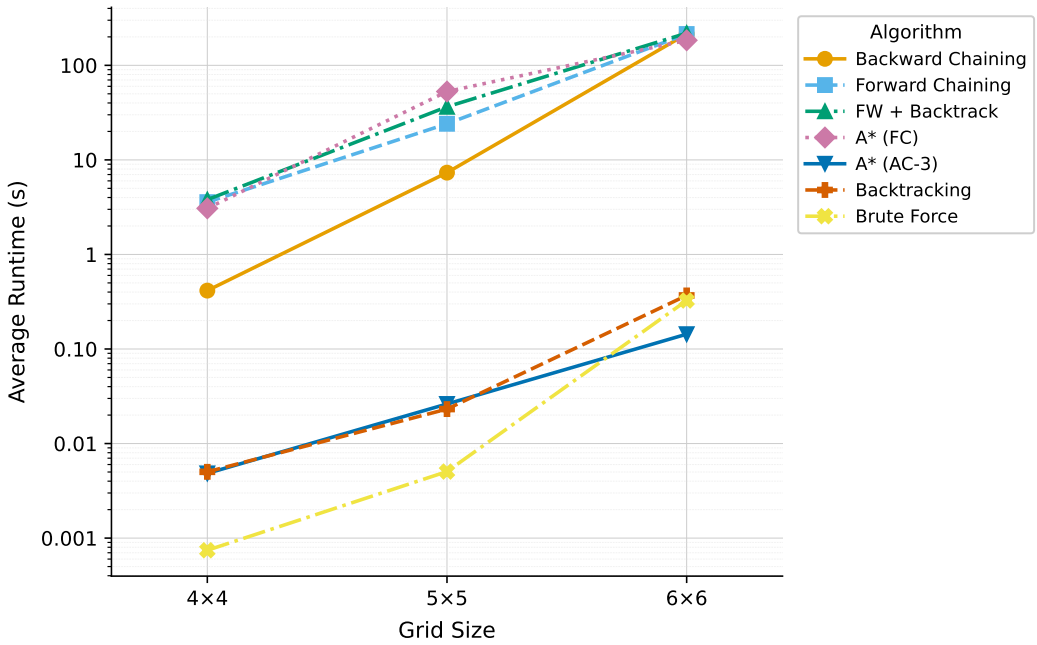


Figure X. Average runtime (log scale) across difficulty levels for 4×4, 5×5, and 6×6 puzzles. A with AC-3 pruning is 2–3 orders of magnitude faster than A* (FC) and FOL-based chains at 6×6. Backward Chaining fails (error) on 6×6 easy and hard; those points are omitted.*

Figure 7.1: Runtime comparison of all algorithms across puzzle sizes and difficulty levels

1. **The Pruner Determines Performance**

   The chart reveals a sharp separation into two tiers: the bottom tier (Brute Force, Backtracking, A* with AC-3) stays below 1 second across all sizes, while the top tier (Backward Chaining, Forward Chaining with/without BT, and A* with FC) ranges from 3–330 seconds, which is approximately three orders of magnitude differences on identical problem instances. The dividing factor is the pruning engine, not the search strategy.

   - Algorithms using AC-3 as their pruning step operate entirely on compact domain arrays. Although the full consistency pass is expensive in theory, it runs in tight numerical loops with very little overhead in practice.
   - Algorithms using the FOL based Forward Chaining engine at every node have to iterate over the grounded rule set and perform pattern unification over the entire fact base, repeated for every candidate value at every cell. A* (h3 + FC) inherits this overhead because it calls the full forward chaining engine at every node expansion, which is why it behaves more like Backward Chaining than like A* (h3 + AC-3).

2. **Constraint Density Inverts the Difficulty Ordering for FOL Algorithms**

- For Backward Chaining, Forward Chaining, and Hybrid BT+FC all of which rely on the FOL unification engine, harder puzzles are frequently solved faster than easier ones at the same grid size. For search based algorithms (Brute Force, Backtracking, A*+AC-3), we observe an opposite trend.
- For unification based algorithms, more constraints mean the `derive_false` function encounters a contradiction after checking fewer premises, because inconsistent value assignments are ruled out almost immediately. A dense constraint graph prunes invalid branches at the root rather than after deep recursion, so the algorithm does very little backtracking work.

7.2.2 Large Grid Scale (7×7)

For the 7×7 benchmark, the FOL based pruning algorithms required too much time to finish within a practical evaluation window. In particular, methods that rely on `fol_derive` at each search step incur very high overhead on large scale instances, so their full runtime statistics are not reported in the chart above. Instead of presenting incomplete.

Table 7.2: FOL based algorithms omitted from detailed 7×7 runtime statistics

Algorithm	**Observation**
Backward Chaining	Execution time was too high on the 7×7 instance, so detailed runtime statistics are omitted.
Forward Chaining	Execution time was too high on the 7×7 instance, so detailed runtime statistics are omitted.
Forward Chaining with Backtracking	Execution time was too high on the 7×7 instance, so detailed runtime statistics are omitted.
A* with FC Pruning	Execution time was too high on the 7×7 instance, so detailed runtime statistics are omitted.

The performance of the remaining algorithms can be observed in the `7x7_runtime` plot.

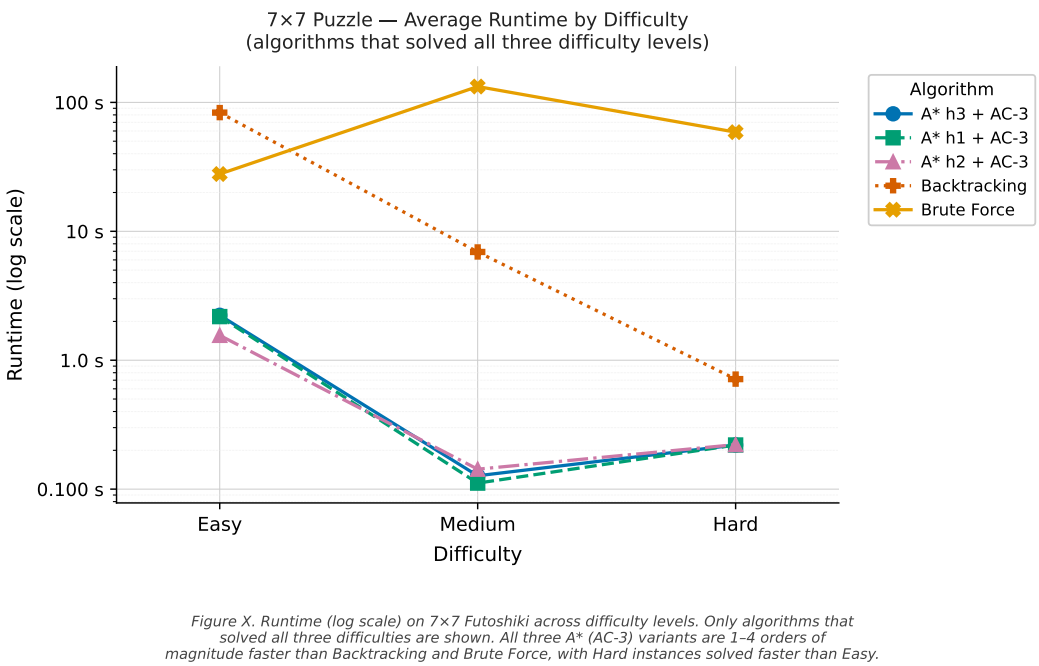


Figure 7.2: Runtime comparison of remaining algorithms on the 7×7 instance

1. **All A* combination outperform Brute Force & Backtracking**

   - All three A*+AC-3 variants operate in the range of 0.11–2.23s. Even the slowest A*+AC-3 result (2.23s for combined+AC-3 on easy) is faster than Backtracking's best result (0.71s on hard) only by a factor of 3, but against Backtracking's worst (83.3s on easy) the gap is 37×, and against Brute Force's worst (132.7s on medium) the gap exceeds 59×.
   - The cause is that Brute Force and Backtracking perform blind enumeration. Brute Force attempts every permutation without constraint awareness; Backtracking prunes only when a direct violation is detected at the current assignment. Neither performs any look ahead or domain propagation. A*+AC-3, by contrast, enforces full arc consistency after every assignment, collapsing many cells to singletons through propagation before the next expansion is even attempted.

7.3 Memory Comparison

7.3.1 Small Grid Scale

This subsection compares the peak memory usage of the evaluated algorithms across different grid sizes and difficulty levels. While runtime highlights search efficiency, memory usage reveals how much state information, intermediate inference data, and

auxiliary structures each method must maintain during solving. Together, these results provide a clearer view of the trade off between speed and space consumption.

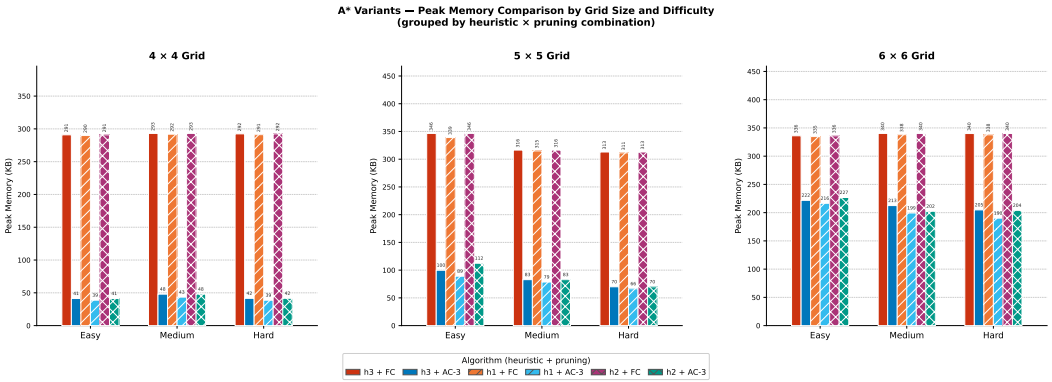


Figure 7.3: Memory comparison of all algorithms on small grid instances

1. **Memory Efficiency Defined by Reasoning Architecture**

   The chart shows a substantial differences that persist across every grid size and difficulty level. Brute Force and Backtracking maintain a minimal memory baseline at 4–11 KB regardless of grid size or difficulty. Forward Chaining, Hybrid BT+FC, A*+FC, and Backward Chaining all cluster in a band of 190–372 KB which roughly 30 to 50 times higher. A*+AC-3 sits alone in a middle tier, starting near 40 KB at 4×4 and rising to 200–222 KB at 6×6.

   - Brute Force and Backtracking store nothing beyond the current partial assignment and their depth first traversal means only the active path is ever in memory. The FOL based algorithms must maintain the full grounded fact base, per recursion binding dictionaries, and domain map copies for isolation across branches. These structures are allocated at initialization and retained throughout.
   - A*+AC-3 falls between the two tiers because it stores domain map copies on the priority queue, more than a single path, but far less than a full fact base. Furthermore, AC-3's compact value set representation is leaner than the tuple based FOL fact store.

2. **A*+AC-3 Reflects Search Behaviour**

   Unlike every other algorithm in the chart, A*+AC-3 shows meaningful memory variation across difficulty levels at the same grid size. At 5×5 it uses 100 KB on easy, 83 KB on medium, and 70 KB on hard. At 6×6 the pattern continues: 222 KB on easy, 213 KB on medium, and 205 KB on hard, a 17 KB spread that mirrors the constraint inversion pattern seen in runtime.

- The cause is that A*+AC-3 stores a full domain map copy for every node currently on the open list, so its peak memory is directly proportional to the maximum frontier size reached during search. Because easy level are under constrained, the search branches more broadly before propagation can compress the state space, so the open list grows and memory rises.
- Hard instances collapse domains aggressively through AC-3 propagation, keeping the frontier narrow and memory low. A*+AC-3 is therefore the only algorithm whose memory profile acts as a direct diagnostic signal for how much genuine search was required.

3. **Backward Chaining Grows for the Opposite Structural Reason**

   Backward Chaining is the one algorithm in the upper tier that breaks the near flat memory pattern. At 4×4 it uses 200 KB on easy, 235 KB on medium, and 194 KB on hard. At 5×5 it uses 321 KB on easy, 318 KB on medium, and 231 KB on hard, showing a large drop from easy to hard at the same grid size.

   - The reason connects directly to the early refutation mechanism from the runtime analysis. On hard instances with dense constraints, `derive_false` detects contradictions after unifying only a few premises, so the recursive `_unify_all` call terminates shallow and its binding copy stack remains thin.
   - On easy instances, the lack of constraints forces the unification engine to traverse much deeper into the fact base before it can confirm or reject a candidate. As a result, more binding dictionaries accumulate on the recursion stack before returning.

7.3.2 Large Grid Scale (7×7)

For the 7×7 benchmark, the FOL based pruning algorithms are not practical for direct comparison. In particular, methods built around `fol_derive` either require too much time to finish or become stuck processing the large FOL fact set, so their detailed memory statistics are omitted from the chart for clarity.

Table 7.3: FOL based algorithms omitted from detailed 7×7 memory statistics

Algorithm	**Observation**
Backward Chaining	Peak memory usage was too high on the 7×7 instance, so detailed memory statistics are omitted.
Forward Chaining	Peak memory usage was too high on the 7×7 instance, so detailed memory statistics are omitted.
Forward Chaining with Backtracking	Peak memory usage was too high on the 7×7 instance, so detailed memory statistics are omitted.
A* with FC Pruning	Peak memory usage was too high on the 7×7 instance, so detailed memory statistics are omitted.

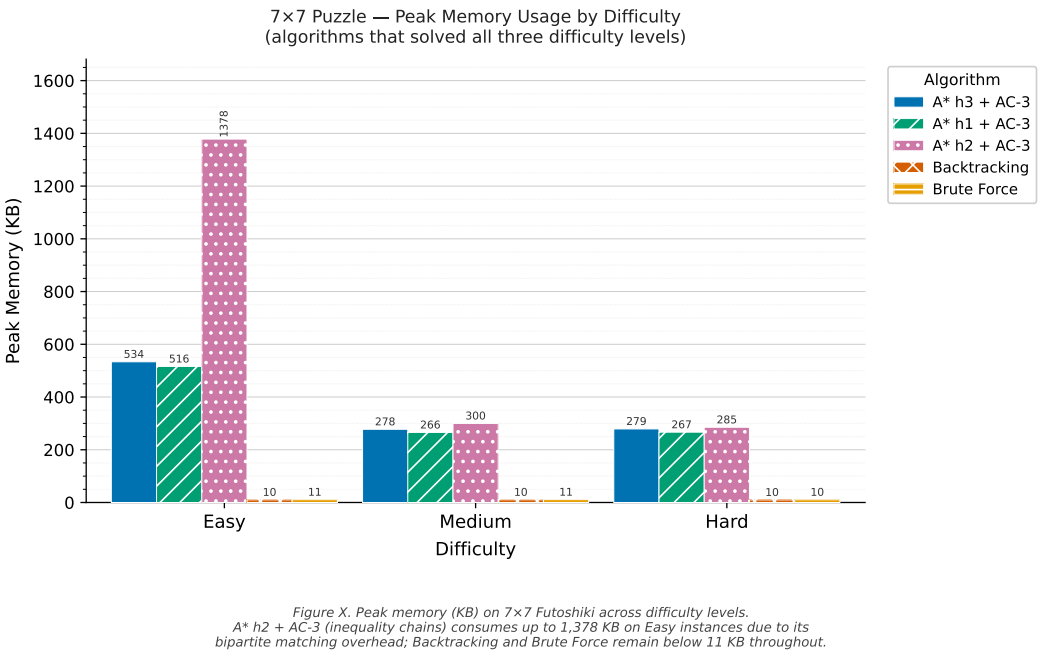


Figure 7.4: Memory comparison of remaining algorithms on the 7×7 instance

1. **Three Memory Tiers Are Set by Data Structures**

   The chart separates into three clear memory tiers. Brute Force and Backtracking stay at the floor, around 10–11 KB. A*+h3 and A*+h1 with AC-3 prunning form a stable middle tier, while A*+h2+AC-3 skyrocket far above them on the easy instance.

   - The key reason is structural, not just procedural. Brute Force and Backtracking store little beyond the current assignment, so their footprint stays

tiny. The A* variants must keep domain map copies on the open list, so they naturally occupy a higher tier.

- The unusual spike for h2 on easy does not come from a different architecture. It comes from the extra matching work inside that heuristic, which adds overhead on top of the same frontier storage used by the other A*+AC-3 variants.

2. **Why A* H3 (Combined Heuristic) Better than H2**

   H3 (combined) tends to use less memory than H2 (`inequality_chains`) because a tighter heuristic makes A* expand fewer nodes, which means fewer successor states are kept on the open list at the same time. Both rely on a bipartite matching lower bound, but H2 is generally weaker than H3's $\max(h1, h2)$ formulation. That tighter estimate guides the search more directly toward the goal and reduces the number of live nodes held in memory simultaneously.

3. **Brute Force and Backtracking Stay Almost Constant**

   Brute Force and Backtracking are the only methods whose memory barely changes across difficulties. Even when runtime changes dramatically, their memory remains almost flat.

   - Both algorithms follow a depth first style and keep only the active assignment path in memory. They do not maintain a wide frontier, a fact base, or extra heuristic structures.
   - As a result, their memory use is tied mostly to grid size rather than puzzle difficulty. In practice, this makes their memory profile nearly constant and far less informative than the A* based methods.

7.4 FC Stall Behavior Investigation

7.4.1 Performance Data

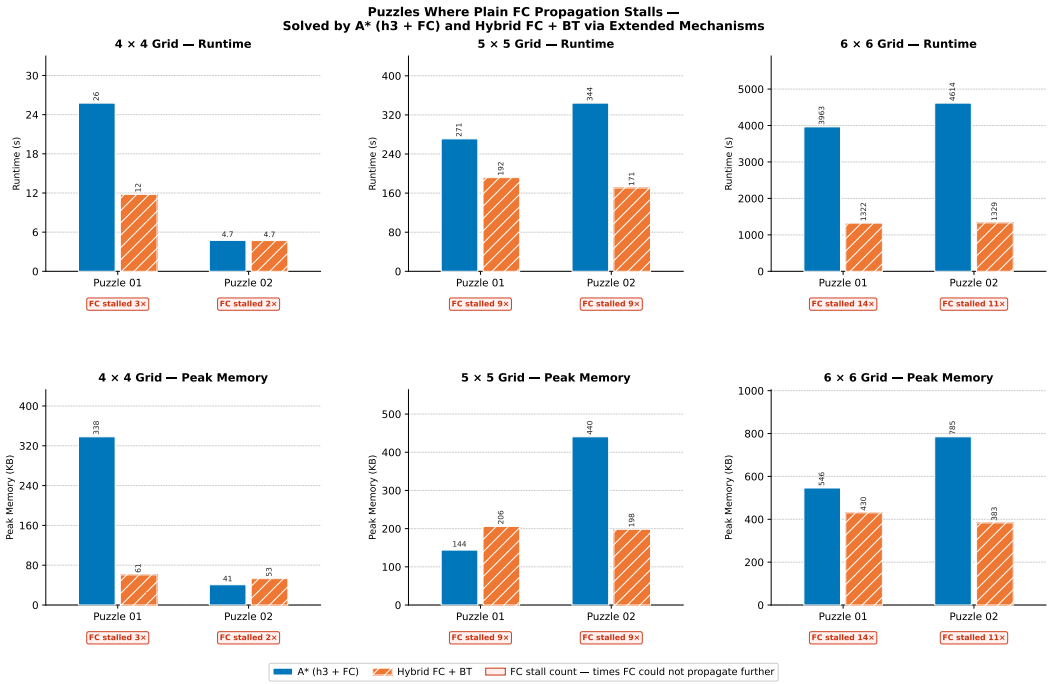


Figure 7.5: Runtime & Memory comparison of FOL based solution for instances that stalled FC

The charts above show runtime and peak memory for the two FC based variants: A* (h_3 + FC) and Hybrid FC+BT on puzzle instances where plain forward chaining propagation stalls. All six instances were selected specifically because standalone FC returns an incomplete solution on them, making them a direct test of how each variant's extended mechanism handles propagation deadlock.

7.4.2 The Stall Problem and How Each Variant Resolves It

Plain forward chaining, as implemented, is a single propagation pass to fixpoint with no search component. When `forward_chaining()` returns `is_complete = False`, the solver marks unsolved cells as zero and exits. No branching, no guessing, no recovery. A stall occurs when every remaining cell retains a domain of size > 1 after all constraint propagation is exhausted, so the algorithm has nothing left to do.

Both variants escape this by using FC as a sub-routine rather than a main solver, which is what allows them to push past these stalls. In Hybrid FC+BT, when FC stalls, the backtracking layer selects the cell with the smallest remaining domain (MRV), makes a tentative assignment, and re invokes FC on the reduced state. If that branch eventually contradicts, backtracking undoes the assignment and tries the next value.

FC handles the deterministic propagation; backtracking supplies the search that FC cannot. In A* (h_3 + FC), the same FC pass runs at every node expansion to propagate consequences before a successor is pushed onto the open list. When FC stalls, A* simply continues expanding nodes from the priority queue. The stall is local to one expansion step, not a terminal condition for the whole algorithm. The stall count is not a failure metric; it is a direct measure of how many cells required a guess before propagation could continue.

7.4.3 Runtime Comparison Between the Two Variants

Hybrid FC+BT is consistently faster than A* (h_3 + FC) across every instance in the chart. At 4×4 Puzzle 01 the gap is 26 s versus 12 s; at 6×6 it widens to 3963–4614 s versus 1322–1329 s, a factor of roughly 3×.

The reason is how each variant responds to a stall. When Hybrid FC+BT stalls, it makes one targeted assignment via MRV and immediately re-runs FC, which may resolve many additional cells in a single propagation pass. The cost of breaking the stall is one assignment plus one FC pass. When A* stalls, it generates one successor per value in the chosen cell's domain, pushes all of them onto the open list, and runs a full FC pass for each. A stall that Hybrid breaks in a single step costs A* an entire round of node generation and redundant FC calls. With 11–14 stalls at 6×6, that difference compounds into the observed 3× gap.

7.4.4 Heuristic Guidance Versus MRV on Stalling Instances

Hybrid FC+BT selects variables by MRV alone. A* (h_3 + FC) adds a global h_3 estimate to order the priority queue, giving it a broader view of which partial states are closer to the goal. But on these instances, that broader view does not help. When FC stalls 9-14 times per puzzle, the cost is dominated by how cheaply each stall is broken, not by node ordering. MRV breaks a stall in one step. h_3 determines which node to expand next but cannot change the fact that every stall forces A* to generate one successor per domain value, run FC on each, and manage the resulting queue insertions. Heuristic ordering matters when the search tree is wide and expansion choices are consequential; here, propagation does most of the work and each stall is a local crisis that MRV resolves more directly than any global estimate can.

7.4.5 Memory Cost of Extended Search

At 4×4, A* uses 338 KB against Hybrid's 61 KB — a 5.5× gap. By 6×6 the ratio narrows to roughly 2×: 546–785 KB for A* versus 383–430 KB for Hybrid.

A* holds a full domain map copy for every open-list node simultaneously. Each stall adds a fresh batch of successors to the heap, so peak memory grows with both grid size and stall count. Hybrid is depth-first and discards state on backtrack, keeping live memory bounded to the current path. However, every backtrack re-invokes FC, which allocates temporary domain copies internally. As stall counts climb from 2–3 at 4×4 to 11–14 at 6×6, those repeated FC allocations accumulate and close the gap, leaving Hybrid with only a 2× advantage where it once had 5.5×.

7.4.6 Improve A* Runtime

A closer inspection of the A* (h_3 + FC) implementation reveals a structural disadvantage that partially explains its runtime disadvantage over Hybrid FC+BT. The current implementation iterates through all values in the chosen cell's domain within the same expansion step: if $\mathrm{Val}(\mathrm{B}, 1)$ stalls FC, it immediately tries $\mathrm{Val}(\mathrm{B}, 2)$, then $\mathrm{Val}(\mathrm{B}, 3)$, exhausting the entire domain before returning control to the queue.

The consequence is that A* loses its global guidance precisely at the moments it needs it most. When FC stalls, the heuristic estimate on the incomplete state may rank it below other open nodes. By forcing local exhaustion instead, the algorithm commits to the current cell regardless of what h_3 would prescribe, effectively degenerating into depth first enumeration at every stall point. The suggested workflow is:

$$\text{A* picks cell B} \rightarrow \mathrm{Val}(\mathrm{B}, 1) \rightarrow \text{FC stalls} \rightarrow \text{push state} \rightarrow \text{queue pop} \rightarrow \text{A* picks cell Q} \rightarrow \mathrm{Val}(\mathrm{Q}, 1) \rightarrow \text{done}$$

The current workflow is:

$$\text{A* picks cell B} \rightarrow \mathrm{Val}(\mathrm{B}, 1) \rightarrow \text{FC stalls} \rightarrow \mathrm{Val}(\mathrm{B}, 2) \rightarrow \text{FC stalls} \rightarrow \mathrm{Val}(\mathrm{B}, 3) \rightarrow \text{FC stalls} \rightarrow \text{queue pop} \rightarrow \ldots$$

Chapter 8

Graphical User Interface

8.1 Design Overview

The Futoshiki puzzle interface is developed using the Python Tkinter library. The layout is divided into three columns to optimize the display of information.

- **Header:** Contains data loading controls, including size, difficulty, and loading a puzzle from a `.txt` file.
- **Left Column:** Houses controls for switching between Play Mode and Algorithm Mode, along with their configurations.
- **Center:** Displays the Futoshiki grid.
- **Right Column:** Provides statistics and a history log.

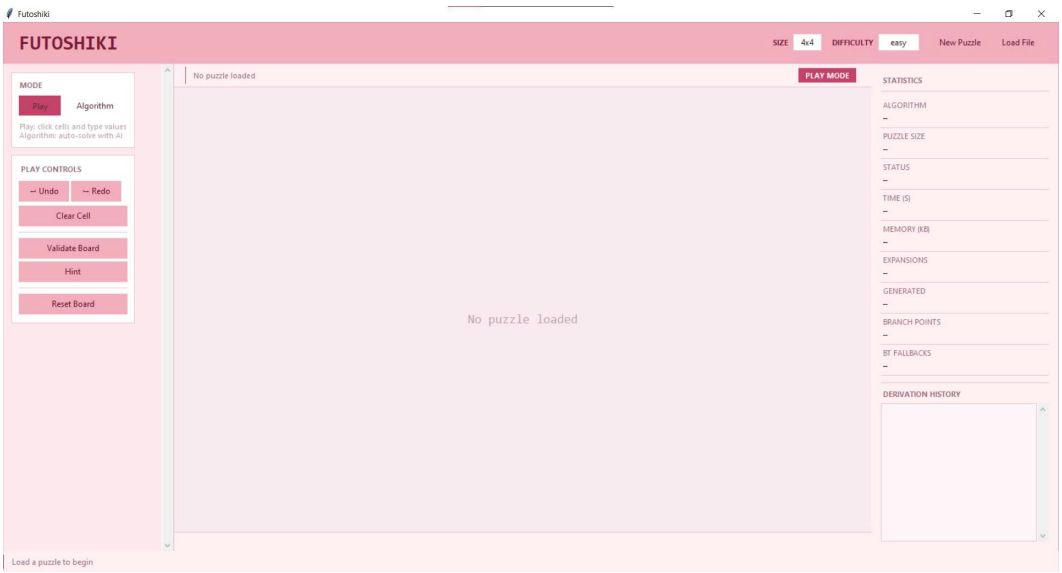


Figure 8.1: The application interface

8.2 Detailed Instructions

Initiate a Puzzle

There are two methods to initiate a puzzle:

- **First method:** Select the grid size and difficulty to load a random puzzle.
- **Second method:** Import a custom puzzle from a structured `.txt` file.

Play Mode

- **Cell interaction:** Click an empty cell and input a number using the keyboard. Some given clues are highlighted and remain uneditable.
- **Validation:** The system performs a quick check. If a value violates a constraint, the background turns red as a warning.
- **Hint:** The system suggests a next step.
- **Undo/Redo:** Allows the user to backtrack or re-apply their inputs.

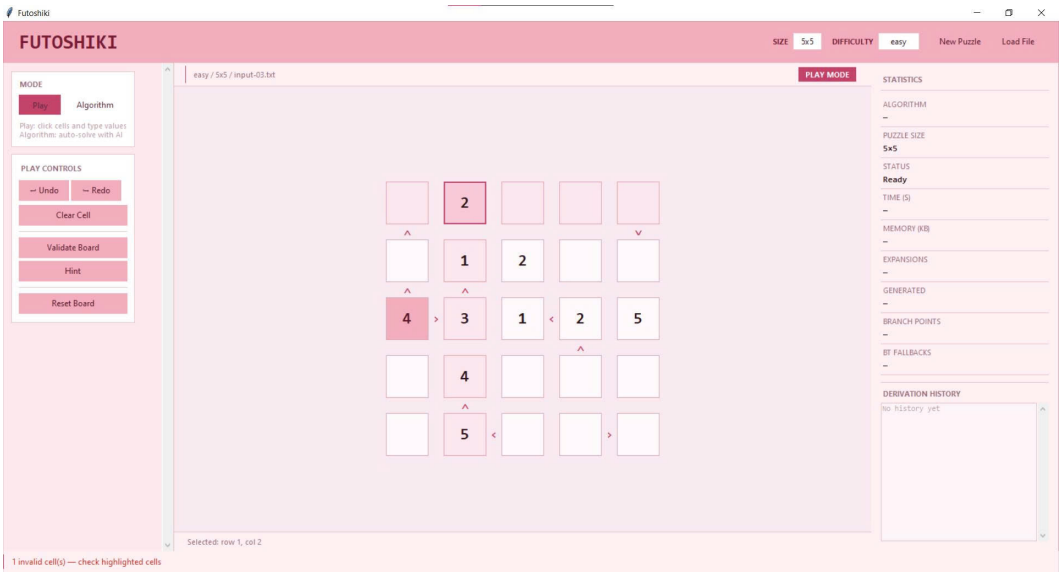


Figure 8.2: The grid during play

Algorithm Mode

- **Algorithm Selection:** Users can choose between A*, Backward Chaining, Forward Chaining, Backtracking, Brute Force, or a hybrid approach.
- **Configuration:** For the A* algorithm, users can customize the heuristic function and pruning techniques, such as FC or AC3.

- **Replay Controls:** After solving, users can watch a step-by-step animation of the process.
  - **Play/Pause:** Automates the step-by-step animation.
  - **Step:** Manually advances the animation one step at a time.
  - **Replay Speed:** Adjusts the animation speed from 0.25× to 8×.

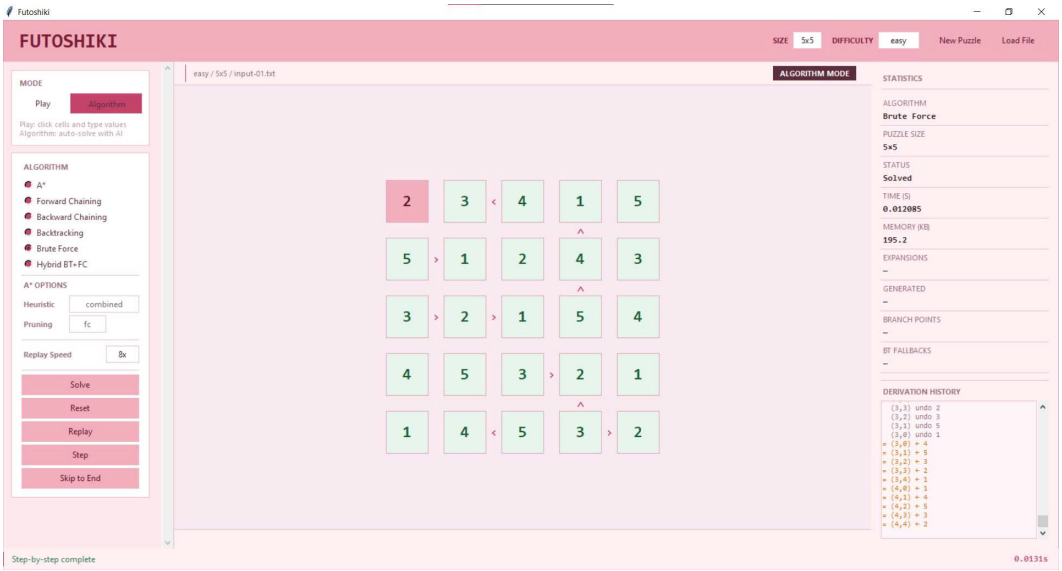


Figure 8.3: The grid after solved

Statistics and History

- **Statistics Panel:** Displays execution time, peak memory usage, and the number of expanded or generated nodes.
- **History:** A scrolled text log that records every action taken by the algorithm.

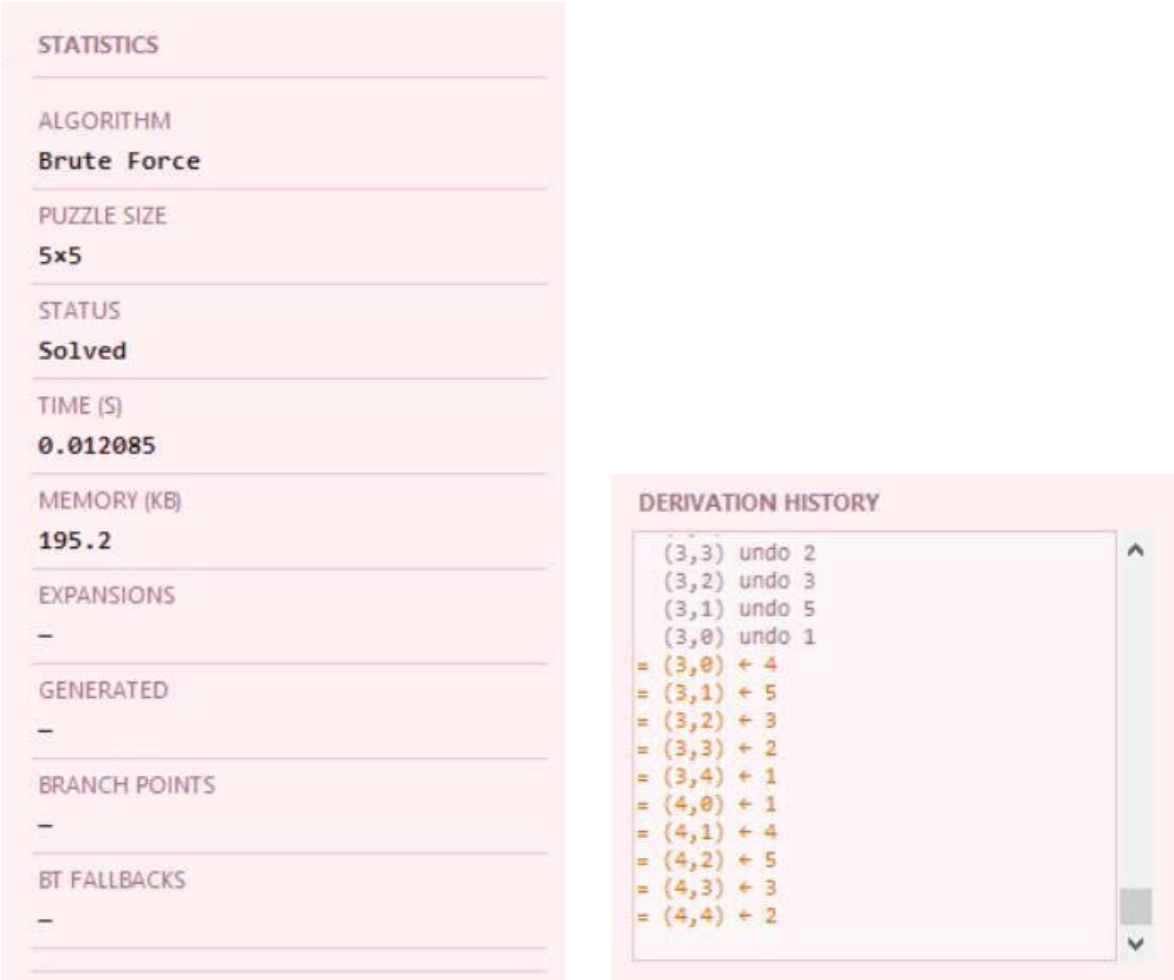


Figure 8.4: Statistics and history log example

Chapter 9

Task Assignment and Self-Evaluation

9.1 Task Assignment

9.1.1 Work Assignment Table

The following table summarizes member information, assigned responsibilities, and completion rates. The workload reflects each member's contribution to the Futoshiki solver, including FOL based reasoning, search design, experiment analysis, interface development, and report preparation.

Table 9.1: Team Member Information and Work Assignment

Member Information	**Assigned Tasks**	**Rate**
Name: Hoàng Trung Hiếu **ID:** 24127041	• Formalized the Futoshiki problem in First-Order Logic. • Implemented the binding and unification mechanisms for the reasoning engine. • Implemented Forward Chaining and integrated it into the solver pipeline. • Helped build the logical backbone connecting symbolic reasoning with search based solving.	100%
Name: Nguyễn Thành Đạt **ID:** 24127021	• Implemented Backward Chaining for the FOL based solver. • Designed and implemented the graphical user interface. • Developed the two interaction modes: `auto_play` (solver mode) and `user_controlled` mode. • Helped connect reasoning modules with the final interactive application.	100%
Name: Lương Hưng Phát **ID:** 24127298	• Implemented A* search and designed the heuristic functions. • Analyzed heuristic behavior and pruning effects across puzzle sizes. • Conducted experiment execution, benchmark testing, and comparison analysis. • Contributed the main experimental interpretation for runtime and memory evaluation.	100%
Name: Nguyễn Minh Khoa **ID:** 24127188	• Contributed to the Forward Chaining implementation and validation. • Assisted in A* heuristic design and refinement. • Supported report formatting, structure cleanup, and final LaTeX organization. • Helped align technical presentation between implementation and written analysis.	100%
Joint Tasks	• Built a strong Futoshiki solver backbone combining multiple search algorithms with FOL based reasoning. • Completed a full GUI with both automatic solver and user controlled interaction modes. • Produced a demonstration video showing the key execution steps of each algorithm on representative instances. • Cross reviewed implementation, experiments, and report writing to ensure consistency.	100%

9.1.2 Self-Evaluation of Project Levels

The team has reviewed the project based on the implemented solver capabilities, experimental depth, user facing features, and the completeness of the final report.

One remaining **limitation** is that we have not yet obtained a full statistical analysis for large puzzle instances when using FOL based algorithms. These methods require very long execution time on larger cases, so collecting stable enough runs for meaningful statistics is still impractical within the project timeframe.

Table 9.2: Self-Evaluation of Completion Rate at Each Project Level

Project Level	**Description / Requirements**	**Rate**
FOL Modeling	Successful formalization of Futoshiki constraints in First-Order Logic, including supporting fact structures, bindings, and inference rules.	100%
Solver Core	Implementation of multiple solving methods including Forward Chaining, Backward Chaining, A* with heuristic guided pruning, and normal search algorithms like Bruteforce; Bakctracking.	100%
Experimental Evaluation	Comprehensive runtime, memory, and search-effort evaluation across multiple puzzle sizes and difficulty levels, with comparison analysis between reasoning and search methods.	100%
GUI and Demonstration	Completion of a functional interface with `auto_play` and `user_controlled` modes, together with a video demonstrating key algorithm steps.	100%
Documentation	Full academic report in LaTeX including formalization, algorithm design, complexity discussion, experiments, analysis, and final presentation.	100%

Project Repository: https://github.com/DrHieuALaska/Futoshiki-puzzle
Project Youtube Video: https://www.youtube.com/watch?v=0oM7XwjGWOk

Bibliography

[1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.